

Unidade II

3 PERSISTÊNCIA E CONEXÃO COM BD COM EF CORE

Uma aplicação web torna-se efetivamente útil quando consegue preservar informações para além do instante em que uma requisição é processada. Enquanto os dados permanecem apenas em memória, eles existem de modo transitório, dependentes da execução atual da aplicação e vulneráveis ao encerramento do processo, à reinicialização do servidor ou à perda do estado temporário. A persistência representa a passagem desses dados para um modelo durável, capaz de registrar informações, recuperá-las em momentos posteriores, relacioná-las a outras entidades e mantê-las disponíveis para diferentes interações do sistema. Em aplicações ASP.NET Core, compreender essa passagem é essencial para construir soluções que não apenas respondam a solicitações imediatas, mas sustentem processos reais ao longo do tempo.

No contexto do PetMatch, a persistência deixa de ser um detalhe técnico e passa a fazer parte do próprio significado da aplicação. Um pet cadastrado para adoção precisa continuar disponível após o término da sessão do usuário. As informações sobre espécie, idade aproximada, porte, comportamento, histórico de saúde, localização e situação de adoção precisam ser armazenadas com consistência, pois orientam decisões responsáveis. Da mesma forma, manifestações de interesse, dados de contato, registros de acompanhamento e alterações no estado de disponibilidade não podem depender de estruturas temporárias. O banco de dados torna-se, assim, a memória organizada do sistema, preservando informações que dão continuidade às interações entre pessoas, instituições e animais.

O banco de dados cumpre uma função distinta da memória da aplicação. A memória é rápida e adequada ao processamento imediato, mas não foi projetada para manter o histórico estável de um sistema. O banco de dados, por sua vez, oferece mecanismos de armazenamento, consulta, relacionamento, integridade e controle de transações. Ele permite que a aplicação represente dados de forma estruturada e confiável, evitando que informações essenciais sejam perdidas ou se tornem contraditórias. Em um sistema de adoção responsável, essa confiabilidade é decisiva, pois dados incorretos ou inconsistentes podem comprometer a experiência dos usuários e a finalidade social da plataforma.

Ao aproximar uma aplicação orientada a objetos de um banco de dados relacional, surge uma diferença importante de modelos. No código C#, o desenvolvedor trabalha com classes, propriedades, objetos e coleções. No banco de dados relacional, as informações são organizadas em tabelas, colunas, linhas, chaves e relacionamentos. Embora ambos os modelos representem dados, eles não o fazem da mesma maneira. A persistência exige, portanto, uma mediação entre o mundo dos objetos e o mundo das relações. Essa mediação é conhecida como mapeamento objeto-relacional, frequentemente associado à sigla ORM (Object-Relational Mapping), e tem como objetivo permitir que entidades do domínio da aplicação sejam armazenadas e recuperadas em estruturas relacionais sem que o desenvolvedor precise escrever manualmente todas as operações de conversão.

O Entity Framework Core (EF Core) ocupa papel central nesse cenário dentro do ecossistema .NET. Ele permite descrever entidades, relacionamentos e regras de mapeamento de modo integrado ao código da aplicação, tornando possível consultar e modificar dados por meio de objetos C#. Em vez de tratar cada operação como uma sequência explícita de comandos SQL (Structured Query Language), o desenvolvedor trabalha com um modelo conceitual mais próximo da linguagem da aplicação. Isso favorece clareza, produtividade e coesão entre domínio e persistência, especialmente quando o sistema está em crescimento e precisa preservar uma estrutura compreensível. Para o PetMatch, essa abordagem permite que elementos como pets, responsáveis, interesses de adoção e registros associados sejam tratados como conceitos do sistema e não apenas como linhas dispersas em tabelas.

O uso de um ORM, contudo, não elimina a necessidade de compreender o banco de dados. A abstração oferecida pelo Entity Framework Core é poderosa, mas não deve ser confundida com invisibilidade da persistência. Consultas, relacionamentos, carregamento de dados, rastreamento de mudanças e transações continuam produzindo efeitos concretos no banco. Um modelo mal desenhado pode gerar consultas ineficientes, relacionamentos confusos ou dificuldades de manutenção. Por isso, a persistência deve ser estudada como uma área de encontro entre arquitetura de software e modelagem de dados. O desenvolvedor precisa entender tanto a expressividade dos objetos em C# quanto as exigências de integridade, desempenho e durabilidade próprias do banco de dados relacional.

Além do Entity Framework Core, o ecossistema .NET também oferece formas de acesso a dados mais próximas do SQL, como ADO.NET e Dapper. O ADO.NET representa uma camada fundamental de comunicação com bancos relacionais, trabalhando diretamente com conexões, comandos, parâmetros, leitores de dados e transações. O Dapper, por sua vez, é frequentemente descrito como um micro-ORM, pois mantém o SQL escrito de forma explícita, mas facilita a conversão dos resultados para objetos C#.



Observação

O ADO.NET é o conjunto clássico de tecnologias de acesso a dados da plataforma .NET, introduzido no início dos anos 2000 com o .NET Framework como evolução do modelo anterior de acesso a dados usado no ecossistema Microsoft, especialmente o ADO baseado em COM. Sua proposta foi oferecer uma forma padronizada de comunicação entre aplicações .NET e bancos de dados relacionais por meio de objetos como `Connection`, `Command`, `DataReader`, `DataAdapter` e `DataSet`. Historicamente, o ADO.NET nasceu em um momento em que aplicações corporativas precisavam lidar com sistemas distribuídos, XML, serviços web e cenários em que nem sempre fazia sentido manter uma conexão aberta continuamente com o banco.

O Dapper surgiu em um contexto posterior, quando ORMs mais completos, como Entity Framework, já eram comuns, mas alguns projetos precisavam de maior controle sobre o SQL e menor sobrecarga de

abstração. Criado no ambiente do Stack Overflow, tornou-se conhecido como um micro-ORM, pois não tenta esconder o banco de dados nem gerar toda a modelagem automaticamente, mas facilita o mapeamento entre resultados SQL e objetos C#. Na prática, o desenvolvedor continua escrevendo consultas SQL, porém Dapper reduz o trabalho repetitivo de abrir comandos, ler colunas e preencher propriedades manualmente.

Neste tópico, essas alternativas aparecem como referência conceitual para situar o leitor no ecossistema .NET. A implementação prática do PetMatch será feita com EF Core e SQLite, porque o objetivo imediato é introduzir persistência integrada ao modelo da aplicação, migrations, seed, consultas e atualização de dados sem exigir que o iniciante escreva SQL manualmente neste primeiro contato. Assim, ADO.NET e Dapper ajudam a entender o mapa das opções, enquanto o código desenvolvido neste momento se concentra no EF Core.

No PetMatch, a persistência deve sustentar a continuidade da experiência do usuário e a responsabilidade sobre os dados cadastrados. A aplicação precisa manter um registro confiável dos animais disponíveis, refletir mudanças de estado, permitir consultas coerentes e preparar o caminho para funcionalidades mais elaboradas.



Lembrete

Ao estudar banco de dados, mapeamento objeto-relacional e alternativas de acesso a dados, o leitor passa a compreender que a camada de persistência participa ativamente da qualidade do sistema, pois influencia a consistência das informações, a clareza do domínio, o desempenho das páginas e a capacidade de evolução da aplicação.

Este tópico introduz, portanto, uma etapa decisiva na construção de aplicações web com ASP.NET Core. Depois de compreender o percurso da requisição e a organização de páginas com MVC e Razor, torna-se necessário tratar os dados como parte durável do sistema. Persistir informações significa transformar interações momentâneas em registros confiáveis, capazes de sobreviver ao fluxo imediato das requisições e de sustentar decisões futuras. Essa passagem da memória para o banco de dados marca a transição de uma aplicação meramente demonstrativa para uma solução com continuidade, histórico e compromisso com a integridade das informações.

3.1 Modelagem, DbContext, migrations, seed e pooling de conexões

A persistência muda a natureza da aplicação. Enquanto os dados permanecem apenas em memória, cada execução do projeto recria o estado inicial e descarta os cadastros feitos durante o uso. Esse comportamento é adequado nos primeiros passos de uma aplicação, porque reduz dependências externas e permite concentrar a atenção em controllers, views, endpoints e validação. Entretanto, um sistema como o PetMatch precisa evoluir para registrar pets de forma durável. A mudança pode ser resumida como uma troca da fonte principal dos dados. Antes, o PetMatch usava `PetMemoryStore`, uma lista em memória criada a cada execução. Agora, passa a usar `PetMatchDbContext`, que representa a conexão entre a aplicação e o banco SQLite. Antes, cadastrar um pet durante a execução produzia um resultado temporário. Agora, o cadastro é gravado em `petmatch.db` e pode continuar disponível depois que a aplicação for encerrada e iniciada novamente.

Quadro 6

Antes	Agora
<code>PetMemoryStore</code>	<code>PetMatchDbContext</code>
Lista em memória	Tabela <code>Pets</code> no SQLite
Dados reiniciam ao executar o projeto	Dados permanecem em <code>petmatch.db</code>
Cadastro serve à sessão atual	Cadastro sobrevive à reinicialização
API e MVC leem uma coleção temporária	API e MVC leem a mesma base persistente

Isso significa que os dados não podem desaparecer quando a aplicação é encerrada. Eles precisam ser gravados em um banco, consultados novamente em execuções futuras e organizados de modo coerente com as regras do sistema.

No EF Core, essa organização começa pela entidade. Uma entidade persistente representa algo que possui identidade própria e deve existir além de uma requisição isolada. No PetMatch, o pet deixa de ser apenas um item dentro de uma lista temporária em memória e passa a ser um registro armazenado em um banco SQLite. Essa entidade não deve ser confundida com o `PetCreateViewModel`. O `ViewModel` representa os dados recebidos pelo formulário de cadastro, ou seja, aquilo que vem da tela no momento em que o usuário preenche as informações. A entidade `Pet`, por sua vez, representa o dado que será salvo e consultado pela aplicação. Essa separação ajuda a manter clara a diferença entre os dados usados para entrada na interface e os dados que fazem parte do estado permanente do sistema.

O `DbContext` é o ponto central de integração entre a aplicação e o EF Core. Ele pode ser entendido como o componente responsável por conversar com o banco de dados durante uma operação. É nele que a aplicação declara quais entidades serão armazenadas e como elas serão acessadas. Por meio de propriedades do tipo `DbSet<T>`, o contexto expõe conjuntos de entidades, como se cada conjunto representasse uma coleção de registros de determinado tipo. No PetMatch, o `PetMatchDbContext` passa a concentrar o acesso persistente aos pets. Com isso, o controller MVC, os endpoints Minimal API e a inicialização da aplicação deixam de depender de uma lista em memória como fonte principal e passam a consultar a mesma base SQLite.

Para introduzir o EF Core, o projeto `PetMatch.Web` recebeu os pacotes `Microsoft.EntityFrameworkCore.Sqlite 10.0.9`, `Microsoft.EntityFrameworkCore.Design 10.0.9` e `SQLitePCLRaw.lib.e_sqlite3 3.50.3`. A escolha por SQLite mantém o exemplo local e simples, sem exigir a instalação ou configuração de um servidor de banco de dados separado. O arquivo `appsettings.json`, no projeto `PetMatch.Web`, recebeu uma connection string para a base do `PetMatch`. A connection string é o texto de configuração que informa à aplicação onde está o banco de dados e quais opções devem ser usadas para acessá-lo.

```
{
  "ConnectionStrings": {
    "PetMatchConnection": "Data Source=petmatch.db;Pooling=True"
  }
}
```

Nessa configuração, `Data Source=petmatch.db` indica que o banco será armazenado em um arquivo chamado `petmatch.db`. Já a opção `Pooling=True` está relacionada ao reaproveitamento de conexões com o banco. De forma simplificada, uma conexão é o canal usado pela aplicação para enviar comandos e receber respostas do banco de dados. Abrir e fechar conexões repetidamente pode gerar custo desnecessário. O pooling de conexões permite que conexões já criadas sejam reaproveitadas quando possível, em vez de criar uma conexão completa para cada operação.

Além disso, o projeto usa `AddDbContextPool` para registrar o `PetMatchDbContext`. Esse recurso também envolve reaproveitamento, mas em outro nível. Enquanto o pooling de conexões trata do reaproveitamento das conexões usadas pelo provedor SQLite, o pooling de contexto trata do reaproveitamento controlado de instâncias do `DbContext` pelo ASP.NET Core. Em outras palavras, o primeiro está ligado ao canal de comunicação com o banco; o segundo está ligado ao objeto do EF Core que organiza o trabalho da aplicação com os dados.

No projeto `PetMatch.Web`, o arquivo `Program.cs` foi ajustado para registrar o contexto com SQLite, manter MVC e Minimal APIs, atualizar a estrutura do banco na inicialização e inserir dados iniciais quando necessário. O trecho a seguir mostra o papel do EF Core na composição da aplicação.

```
using Microsoft.EntityFrameworkCore;
using PetMatch.Web.Data;
using PetMatch.Web.Endpoints;
using PetMatch.Web.Filters;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContextPool<PetMatchDbContext>(options =>
{
    var connectionString =
builder.Configuration.GetConnectionString("PetMatchConnection");
    options.UseSqlite(connectionString);
});

builder.Services.AddScoped<PetMatchActionFilter>();
```

```
builder.Services.AddControllersWithViews(options =>
{
    options.Filters.AddService<PetMatchActionFilter>();
});

var app = builder.Build();

using (var scope = app.Services.CreateScope())
{
    var dbContext = scope.ServiceProvider.GetRequiredService<PetMatchDbContext>();
    dbContext.Database.Migrate();

    var seed = scope.ServiceProvider.GetRequiredService<PetMatchSeed>();
    seed.Seed();
}

app.UseHttpsRedirection();
app.UseDefaultFiles();
app.UseStaticFiles();

app.MapPetEndpoints();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller}/{action=Index}/{id?}");

app.Run();
```

Esse código demonstra a mudança estrutural da aplicação. O armazenamento em memória deixa de ser a principal forma de guardar os dados e o `PetMatchDbContext` passa a ser configurado para usar SQLite. A chamada `dbContext.Database.Migrate()` verifica se o banco local já está de acordo com a estrutura esperada pela aplicação. Caso existam mudanças registradas pelo EF Core, elas são aplicadas ao banco. Essas mudanças são chamadas de migrations. Uma migration funciona como um histórico controlado das alterações feitas no modelo de dados, como a criação de uma tabela, a inclusão de uma coluna ou a alteração de uma configuração. Assim, em vez de modificar o banco manualmente, a aplicação consegue atualizar sua estrutura de forma organizada.

Depois disso, o código obtém uma instância de `PetMatchSeed` e chama o método `Seed()`. O termo `seed` significa a inserção de dados iniciais. Esses dados servem para que a aplicação comece com informações mínimas, úteis para testes, demonstrações ou funcionamento básico. No contexto do `PetMatch`, o `seed` pode garantir que existam pets previamente cadastrados quando a aplicação for executada pela primeira vez, evitando que o usuário encontre uma base completamente vazia.

Com essas alterações, o `PetMatch` passa a ter uma base de dados real e durável, sem perder a simplicidade do exemplo local. As rotas continuam preservadas, como `/`, `/Pets`, `/Pets/Create`, `/api/pets`, `/api/pets/{id}`, `/api/boas-vindas/{nome}` e `/api/interesses`. A diferença principal está na fonte dos dados: antes, eles viviam apenas temporariamente em memória; agora, são armazenados em SQLite e acessados por meio do EF Core.

O arquivo `Data/PetMatchDbContext.cs`, no projeto `PetMatch.Web`, define o contexto do EF Core. Ele expõe `Pets` como `DbSet<Pet>` e configura a entidade persistente, incluindo o mapeamento das características usadas nos cards.

```
using Microsoft.EntityFrameworkCore;
using PetMatch.Web.Models;

namespace PetMatch.Web.Data;

public sealed class PetMatchDbContext : DbContext
{
    public PetMatchDbContext(DbContextOptions<PetMatchDbContext> options)
        : base(options)
    {
    }

    public DbSet<Pet> Pets => Set<Pet>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Pet>(entity =>
        {
            entity.HasKey(pet => pet.Id);

            entity.Property(pet => pet.Nome).HasMaxLength(80).IsRequired();
            entity.Property(pet => pet.Especie).HasMaxLength(40).IsRequired();
            entity.Property(pet => pet.Porte).HasMaxLength(30).IsRequired();
            entity.Property(pet => pet.Cidade).HasMaxLength(80).IsRequired();
            entity.Property(pet => pet.Descricao).HasMaxLength(300).IsRequired();
            entity.Property(pet => pet.Icone).HasMaxLength(12).IsRequired();
            entity.Property(pet => pet.CaracteristicasTexto).HasMaxLength(300)
                .IsRequired();
        });
    }
}
```

Esse contexto torna explícito que a entidade `Pet` possui regras de armazenamento. Tamanhos máximos, obrigatoriedade de campos e chave primária deixam de ser apenas convenções implícitas. O EF Core usa esse modelo para criar a estrutura do banco e materializar objetos quando os dados são consultados.

A entidade persistente `Pet`, no arquivo `Models/Pet.cs` do projeto `PetMatch.Web`, foi ajustada para funcionar com o EF Core e com as views já existentes. Ela mantém os dados usados pelos cards e preserva uma forma conveniente de expor as características como coleção para a interface.

Essa alteração também explica por que o `Pet` deixa de ser um `record` simples e passa a ser uma `class` com propriedades `get`; `set`; . Nos tópicos anteriores, o `pet` era principalmente um objeto de transporte de dados em memória. Com EF Core, ele passa a ser uma entidade persistente: o framework precisa criar instâncias ao ler linhas do banco, preencher propriedades, acompanhar alterações e decidir o que deve ser gravado em `SaveChangesAsync()`. Propriedades mutáveis tornam esse processo mais direto para o rastreamento de entidades e a materialização dos dados vindos do banco.

```
using System.ComponentModel.DataAnnotations.Schema;

namespace PetMatch.Web.Models;

public sealed class Pet
{
    public int Id { get; set; }

    public string Nome { get; set; } = string.Empty;

    public string Especie { get; set; } = string.Empty;

    public string Porte { get; set; } = string.Empty;

    public int IdadeAproximada { get; set; }

    public string Cidade { get; set; } = string.Empty;

    public string Descricao { get; set; } = string.Empty;

    public string Icone { get; set; } = "🐾";

    public bool Disponivel { get; set; } = true;

    public string CaracteristicasTexto { get; set; } = string.Empty;

    [NotMapped]
    public string[] Caracteristicas
    {
        get => CaracteristicasTexto
            .Split('|', StringSplitOptions.RemoveEmptyEntries |
StringSplitOptions.TrimEntries);

        set => CaracteristicasTexto = string.Join('|', value);
    }
}
```

Esse modelo separa a forma persistida da forma usada pela apresentação. O banco armazena `CaracteristicasTexto` como um texto simples, separando os itens por `|`, enquanto a view continua podendo percorrer `Caracteristicas` como uma coleção. O atributo `[NotMapped]` indica que essa propriedade calculada não será criada como coluna independente no banco. Em uma modelagem mais completa, características poderiam virar outra tabela relacionada a `Pets`, em uma relação um-para-muitos. Aqui, a escolha é didática: preservar os cards já existentes e introduzir persistência sem acrescentar relacionamento entre tabelas no mesmo momento.

As migrations registram a evolução do esquema. No projeto `PetMatch.Web`, foi criada a pasta `Migrations`, com os arquivos `202607030001_CreatePetMatchSchema.cs` e `PetMatchDbContextModelSnapshot.cs`. A migration inicial descreve a criação da tabela de pets a partir do modelo configurado. O snapshot guarda a forma atual do modelo para que o EF Core consiga comparar mudanças de estrutura quando novas alterações forem introduzidas.

O seed garante que a aplicação não abra vazia. No arquivo `Data/PetMatchSeed.cs`, o projeto cria pets iniciais quando a tabela ainda não possui registros. Isso preserva a experiência visual construída anteriormente: ao acessar `/Pets`, o leitor continua vendo cards prontos, agora vindos do banco SQLite.

```
using PetMatch.Web.Models;

namespace PetMatch.Web.Data;

public sealed class PetMatchSeed
{
    private readonly PetMatchDbContext _dbContext;

    public PetMatchSeed(PetMatchDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public void Seed()
    {
        if (_dbContext.Pets.Any())
        {
            return;
        }

        _dbContext.Pets.AddRange(
            new Pet
            {
                Nome = "Luna",
                Especie = "Cachorro",
                Porte = "Médio",
                IdadeAproximada = 3,
                Cidade = "Curitiba",
                Descricao = "Dócil, brincalhona e acostumada com crianças.",
                Icone = "🐕",
                Disponivel = true,
                Caracteristicas = ["Vacinada", "Castrada", "Sociável"]
            },
            new Pet
            {
                Nome = "Mingau",
                Especie = "Gato",
                Porte = "Pequeno",
                IdadeAproximada = 2,
                Cidade = "São Paulo",
                Descricao = "Calmo, curioso e ideal para apartamento.",
                Icone = "🐈",
                Disponivel = true,
                Caracteristicas = ["Vermifugado", "Independente", "Carinhoso"]
            });

        _dbContext.SaveChanges();
    }
}
```

Esse seed é deliberadamente simples. Ele verifica se já existem pets e, caso a tabela esteja vazia, insere registros iniciais. A consequência prática é que o banco nasce com conteúdo suficiente para a listagem MVC, para a home atualizada e os endpoints da API. Como os dados passam a ser persistidos, um cadastro feito em `/Pets/Create` permanece após parar e iniciar novamente a aplicação pelo Visual Studio.

O controller MVC passa a usar o `PetMatchDbContext` para consultar e gravar dados. No projeto `PetMatch.Web`, o arquivo `Controllers/PetsController.cs` mantém as actions `Index`, `Create GET` e `Create POST`, mas a listagem e o cadastro passam a operar sobre o banco.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using PetMatch.Web.Data;
using PetMatch.Web.Models;
using PetMatch.Web.ViewModels;

namespace PetMatch.Web.Controllers;

public sealed class PetsController : Controller
{
    private readonly PetMatchDbContext _dbContext;

    public PetsController(PetMatchDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    [HttpGet]
    public async Task<IActionResult> Index()
    {
        var pets = await _dbContext.Pets
            .Where(pet => pet.Disponivel)
            .OrderBy(pet => pet.Nome)
            .ToListAsync();
        return View(pets);
    }

    [HttpGet]
    public IActionResult Create()
    {
        return View(new PetCreateViewModel());
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(PetCreateViewModel viewModel)
    {
        if (!ModelState.IsValid)
        {
            return View(viewModel);
        }
    }
}
```

```

var pet = new Pet
{
    Nome = viewModel.Nome.Trim(),
    Espécie = viewModel.Espécie.Trim(),
    Porte = viewModel.Porte.Trim(),
    IdadeAproximada = viewModel.IdadeAproximada,
    Cidade = viewModel.Cidade.Trim(),
    Descrição = viewModel.Descrição.Trim(),
    Ícone = viewModel.Ícone.Trim(),
    Disponível = true,
    Características = viewModel.Características
        .Split(',',
StringSplitOptions.RemoveEmptyEntries |
StringSplitOptions.TrimEntries)
    };

_dbContext.Pets.Add(pet);
await _dbContext.SaveChangesAsync();

TempData["MensagemSucesso"] = $"Cadastro de {pet.Nome} criado com
sucesso.";

return RedirectToAction(nameof(Index));
}
}

```

Esse controller mostra a relação entre MVC e persistência. A action `Index` consulta o banco e entrega os pets para a view. A action `POST` continua respeitando o `ModelState`, mas agora cria uma entidade persistente chamada `SaveChangesAsync()`. A página não muda de finalidade; o que muda é a durabilidade do dado gravado pelo formulário.

Os endpoints Minimal API também passam a ler a mesma base persistente. No arquivo `Endpoints/PetEndpoints.cs`, os caminhos públicos foram preservados, mas as consultas usam `PetMatchDbContext` (figura 14). Dessa forma, a listagem MVC, a home e `/api/pets` passam a refletir os mesmos pets cadastrados.

```

api.MapGet("/pets", async (
    PetMatchDbContext dbContext,
    string? especie,
    string? porte,
    string? cidade) =>
{
    var consulta = dbContext.Pets
        .Where(pet => pet.Disponível)
        .AsQueryable();

    if (!string.IsNullOrWhiteSpace(especie))
    {
        consulta = consulta.Where(pet => pet.Espécie == especie);
    }
}

```

```

if (!string.IsNullOrWhiteSpace(porte))
{
    consulta = consulta.Where(pet => pet.Porte == porte);
}

if (!string.IsNullOrWhiteSpace(cidade))
{
    consulta = consulta.Where(pet => pet.Cidade.Contains(cidade));
}

var pets = await consulta
    .OrderBy(pet => pet.Nome)
    .ToListAsync();

return Results.Ok(pets);
});

```

Esse trecho demonstra a principal consequência arquitetural do subtópico: MVC e Minimal APIs deixam de consultar fontes diferentes. O PetMatch passa a ter uma base persistente compartilhada. A home foi atualizada com botões para /Pets e /Pets/Create, e a API continua disponível para consulta em JSON. O resultado visual esperado é a home com acesso claro à área MVC, /Pets exibindo cards vindos do banco, /Pets/Create mantendo o formulário estilizado e /api/pets refletindo os mesmos dados persistidos.

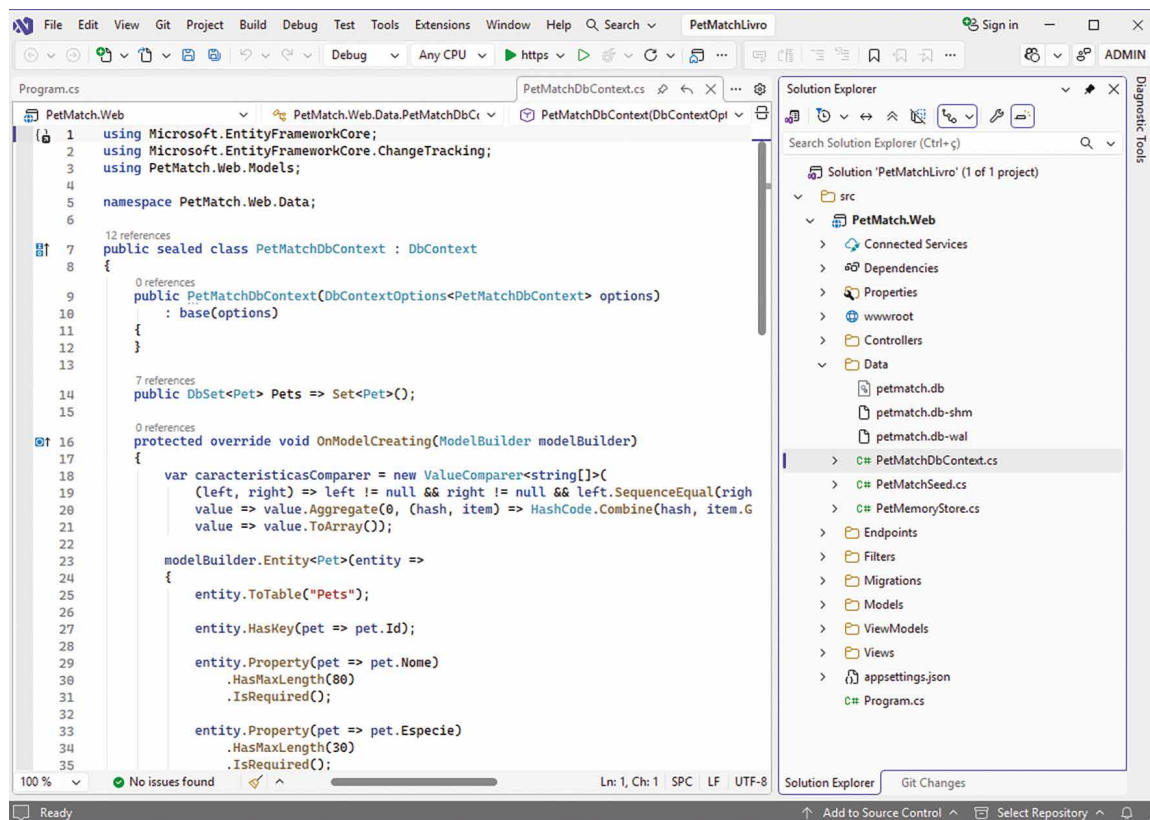


Figura 14 – Arquivo PetMatchDbContext no Visual Studio 2026. À direita está o diretório Data

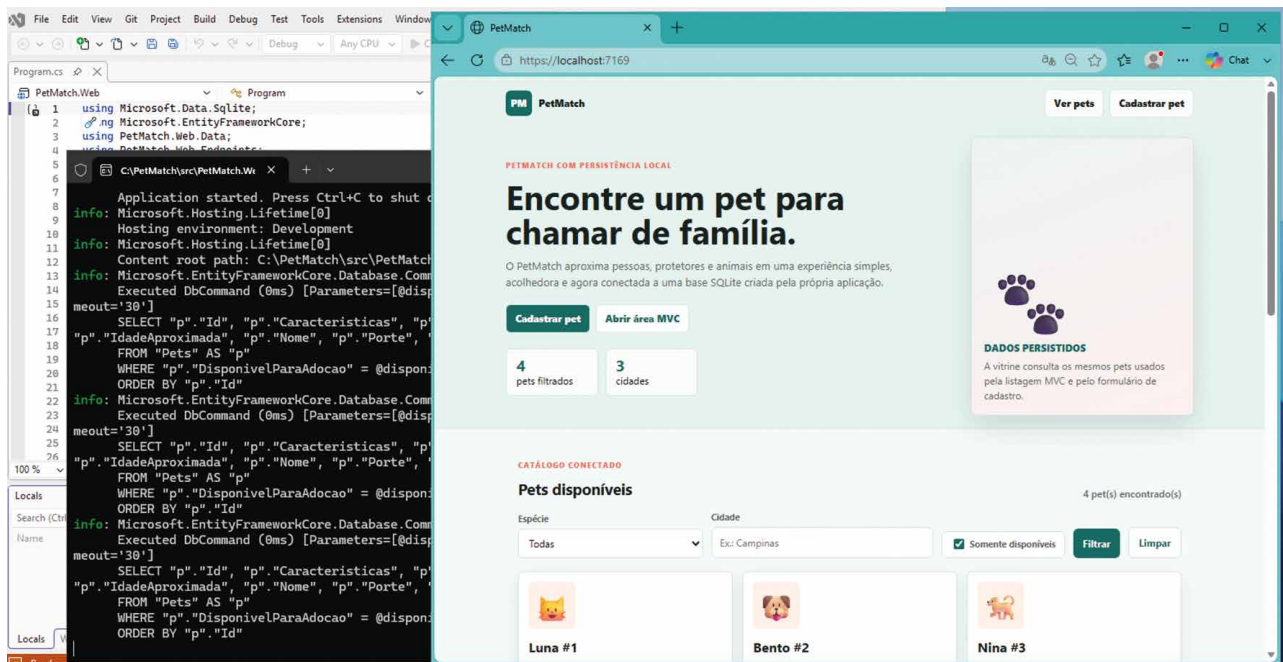


Figura 15 – À direita (em verde), página home do PetMatch. À esquerda, tela do Console (fundo preto) com as consultas executadas durante a execução no Visual Studio 2026 (detalhe parcial)

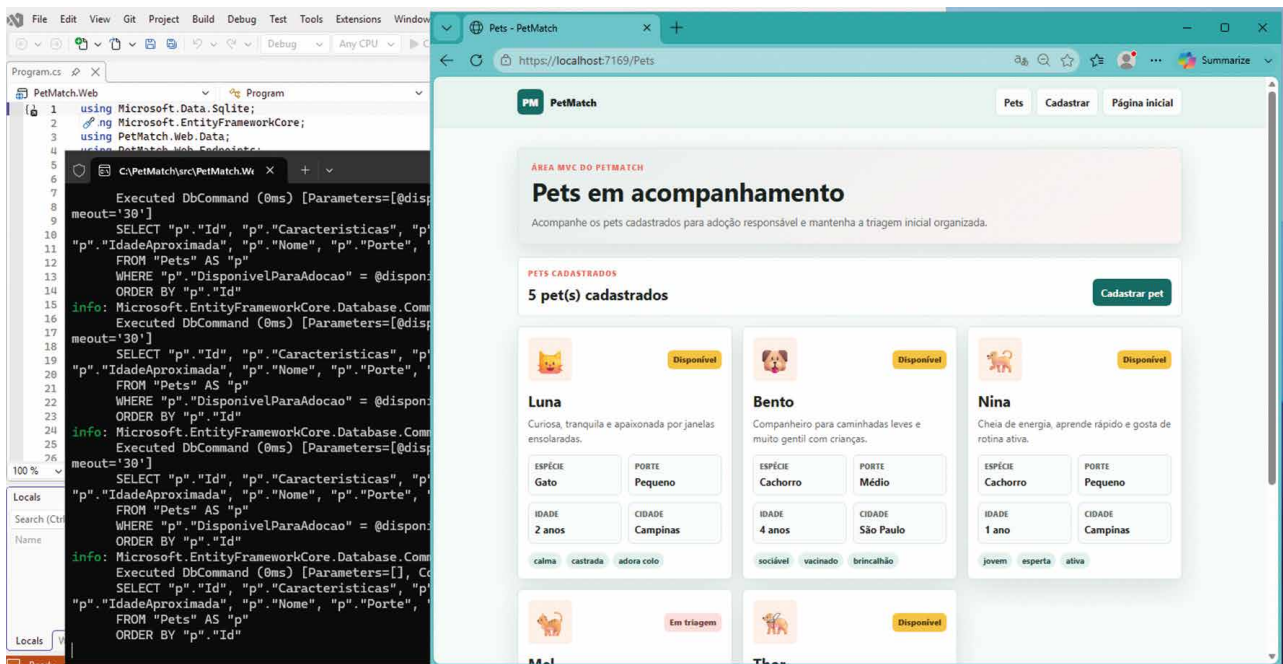


Figura 16 – À direita (em verde), página do PetMatch via /Pets. À esquerda, tela do Console (fundo preto) com as consultas executadas durante a execução no Visual Studio 2026 (detalhe parcial)

Cadastrar pet

Registre dados claros para apresentar o pet na listagem e apoiar a triagem de adoção.

DADOS DO PET

Dados para triagem

Preencha informações objetivas para que a equipe acompanhe cada pet com cuidado.

Nome: Theo

Espécie: Cachorro

Porte: Pequeno

Idade aproximada: 1

Cidade: Santana de Parnaíba

Ícone: Cão jovem

Descrição: Fofinho, alegre e bem cuidado

Características: calmo, vacinado e bagunçeiro

☒ Disponível para adoção

Cadastrar pet **Voltar para listagem**

PetMatch

adoção responsável

Figura 17 – Cadastro de um novo pet (Theo) via /Pets/Create

Visual Studio 2022 Console (Left):

```

1 using Microsoft.Data.Sqlite;
2 using Microsoft.EntityFrameworkCore;
3 using PetMatch.Web.Data;
4
5
6
7
8 ORDER BY "p"."Id"
9 Info: Microsoft.EntityFrameworkCore.Database.Command: Executed DbCommand (0ms) [Parameters=[@disponivel=1], CommandTimeout=30]
10 SELECT "p"."Id", "p"."Características", "p"."IdadeAproximada", "p"."Nome", "p"."Porte", "p"."Cidade"
11 FROM "Pets" AS "p"
12 WHERE "p"."DisponivelParaAdocao" = @disponivel
13 ORDER BY "p"."Id"
14
15 Info: Microsoft.EntityFrameworkCore.Database.Command: Executed DbCommand (0ms) [Parameters=[@p0=1], CommandTimeout=30]
16 INSERT INTO "Pets" ("Características", "Cidade", "IdadeAproximada", "Nome", "Porte", "Symbol")
17 VALUES (@p0, @p1, @p2, @p3, @p4, @p5)
18 RETURNING "Id"
19 Info: Microsoft.EntityFrameworkCore.Database.Command: Executed DbCommand (0ms) [Parameters=[@p0=1], CommandTimeout=30]
20 SELECT "p"."Id", "p"."Características", "p"."IdadeAproximada", "p"."Nome", "p"."Porte", "p"."Cidade"
21 FROM "Pets" AS "p"
22 ORDER BY "p"."Id"

```

PetMatch Web (Right):

Luna (Disponível): Gato, Pequeno, 2 anos, Campinas, calmo, castrada, adora colo.

Bento (Disponível): Cachorro, Médio, 4 anos, São Paulo, sociável, vacinado, brincalhão.

Nina (Disponível): Cachorro, Pequeno, 1 ano, Campinas, jovem, esperta, ativa.

Mel (Em triagem): Gato, Médio, 3 anos, Sorocaba, observadora, independente, em triagem.

Thor (Disponível): Cachorro, Grande, 5 anos, Jundiaí, calmo, leal, porte grande.

Theo (Disponível): Cachorro, Pequeno, 1 ano, Santana de Parnaíba, calmo, vacinado e bagunçeiro.

PetMatch

adoção responsável

Figura 18 – À direita (em verde), página do PetMatch via /Pets, mostrando o card do novo pet cadastrado (Theo), após consulta na base de dados. À esquerda, tela do Console (fundo preto) com as consultas executadas durante a execução no Visual Studio 2026 (detalhe parcial)

3.2 Transações/concorrência (RowVersion, DbTransaction), leituras e comandos eficientes

A persistência introduz um problema que não aparece quando os dados existem apenas em memória durante a execução: a aplicação passa a lidar com alterações duráveis, compartilhadas e potencialmente concorrentes. Em um sistema como o PetMatch, dois atendentes podem abrir o cadastro do mesmo pet, revisar informações e tentar salvar mudanças em momentos próximos. Se a aplicação simplesmente aceitar a última gravação, uma alteração pode sobrescrever outra sem que o usuário perceba. Por isso, persistir dados com EF Core exige mais do que criar tabelas e chamar `SaveChangesAsync`; exige pensar em transações, concorrência e eficiência.



Observação

Uma transação é uma unidade de trabalho que deve ser concluída por inteiro ou desfeita por inteiro. Em EF Core, uma chamada comum a `SaveChangesAsync()` já usa transação quando necessário para preservar a consistência daquele salvamento. Portanto, um cadastro ou uma edição simples nem sempre exige que o desenvolvedor abra uma transação manualmente. Neste tópico, `BeginTransactionAsync()` aparece com finalidade didática: tornar visível o conceito de iniciar, confirmar ou desfazer uma unidade de trabalho. No PetMatch, esse controle é mostrado no fluxo de edição para que o leitor veja a relação entre alteração de dados, renovação do token de concorrência e confirmação da gravação.

A concorrência otimista parte da ideia de que conflitos podem acontecer, mas não são frequentes o suficiente para justificar bloqueios longos. Em vez de bloquear o pet enquanto uma pessoa edita o formulário, a aplicação grava um token de versão no momento em que a tela é carregada. Ao salvar, o EF Core compara o token recebido do formulário com o token atual do banco. Se outro fluxo alterou o mesmo pet nesse intervalo, os tokens deixam de coincidir e o EF Core lança uma exceção de concorrência. A aplicação, então, devolve uma mensagem compreensível em vez de sobrescrever silenciosamente os dados.

A situação pode ser imaginada em três momentos. Primeiro, o usuário A abre `/Pets/Edit/4` e recebe os dados atuais do pet, junto com uma versão do registro. Depois, o usuário B altera o mesmo pet e salva antes do usuário A. Nesse momento, a versão do banco muda. Por fim, o usuário A tenta salvar o formulário antigo. Como ele está enviando uma versão que não é mais a atual, o EF Core identifica o conflito e a aplicação mostra uma mensagem pedindo revisão dos dados.

Em bancos como SQL Server, `rowversion` pode ser uma coluna gerenciada automaticamente pelo próprio banco. O PetMatch usa SQLite local, que não possui esse recurso nativo com a mesma semântica. Por isso, neste projeto, `RowVersion` é um token de concorrência gerenciado pela aplicação, ele é armazenado como `byte[]`, configurado no EF Core como `concurrency token` e renovado pelo código antes de salvar uma atualização. O nome preserva o conceito, mas o funcionamento respeita o provedor usado.

No projeto `PetMatch.Web`, o arquivo `Models/Pet.cs` foi alterado para incluir o token de concorrência. Esse trecho mostra a propriedade adicionada à entidade persistente.

```
public byte[] RowVersion { get; set; } = Guid.NewGuid().ToByteArray();
```

Essa propriedade passa a acompanhar cada pet persistido. Quando um formulário de edição é aberto, o valor atual de `RowVersion` é enviado para a view em formato transportável. Quando o usuário salva, esse valor volta para o controller, permitindo que o EF Core compare a versão originalmente lida com a versão existente no banco no momento da gravação.

No arquivo `Data/PetMatchDbContext.cs`, também no projeto `PetMatch.Web`, a propriedade é configurada como token de concorrência. A configuração pertence ao mapeamento da entidade `Pet`.

```
entity.Property(pet => pet.RowVersion)
    .IsConcurrencyToken()
    .IsRequired();
```

Esse mapeamento informa ao EF Core que `RowVersion` deve participar da verificação de concorrência. Em uma atualização, o valor original do token é incluído na condição usada para localizar o registro a ser alterado. Se nenhum registro corresponder ao identificador e à versão original, o EF Core entende que houve conflito e sinaliza o problema por meio de `DbUpdateConcurrencyException`.

A mudança de esquema foi registrada pela migration `Migrations/202607030002_AddPetConcurrencyToken.cs`, no projeto `PetMatch.Web`. O trecho essencial da migration adiciona a coluna `RowVersion` à tabela `Pets`.

```
migrationBuilder.AddColumn<byte[]>(
    name: "RowVersion",
    table: "Pets",
    type: "BLOB",
    nullable: false,
    defaultValue: Array.Empty<byte>());
```



Lembrete

Essa migration amplia a tabela existente sem alterar as rotas públicas da aplicação. Como o projeto já aplica migrations durante a inicialização, a nova coluna é criada quando a aplicação é executada pelo Visual Studio 2026. O snapshot do EF Core também é atualizado para refletir o novo estado do modelo.

Para transportar o token entre a view e o controller, criamos no projeto o `ViewModels/PetEditViewModel.cs`. O `ViewModel` mantém os campos editáveis e inclui `RowVersionBase64`, pois um `byte[]` não deve ser colocado diretamente em um formulário HTML.

```
using System.ComponentModel.DataAnnotations;

namespace PetMatch.Web.ViewModels;

public sealed class PetEditViewModel
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Informe o nome do pet.")]
    [StringLength(80, ErrorMessage = "O nome deve ter no máximo 80 caracteres.")]
    public string Nome { get; set; } = string.Empty;

    [Required(ErrorMessage = "Informe a cidade.")]
    [StringLength(80, ErrorMessage = "A cidade deve ter no máximo 80 caracteres.")]
    public string Cidade { get; set; } = string.Empty;

    [Required(ErrorMessage = "Informe a descrição.")]
    [StringLength(300, ErrorMessage = "A descrição deve ter no máximo
300 caracteres.")]
    public string Descricao { get; set; } = string.Empty;

    public bool Disponivel { get; set; } = true;

    public string RowVersionBase64 { get; set; } = string.Empty;
}
```

Esse ViewModel mostra a diferença entre edição de tela e entidade persistente. O formulário não precisa expor todos os detalhes internos de `Pet`, mas precisa transportar o token de concorrência. A conversão para Base64 permite que o valor viaje em um campo oculto sem perder sua forma binária original.

No arquivo `Controllers/PetsController.cs`, a action `GET` de edição carrega o pet com `AsNoTracking`. Esse método é apropriado para leituras em que a entidade será usada apenas para montar a tela, sem necessidade de rastreamento imediato pelo contexto.

```
[HttpGet]
public async Task<IActionResult> Edit(int id)
{
    var pet = await _dbContext.Pets
        .AsNoTracking()
        .FirstOrDefaultAsync(pet => pet.Id == id);

    if (pet is null)
    {
        return NotFound();
    }

    var viewModel = new PetEditViewModel
    {
        Id = pet.Id,
        Nome = pet.Nome,
        Cidade = pet.Cidade,
        Descricao = pet.Descricao,
```

```
        Disponível = pet.Disponível,  
        RowVersionBase64 = Convert.ToBase64String(pet.RowVersion)  
    };  
  
    return View(viewModel);  
}
```

O controller busca apenas o pet necessário, não mantém a entidade rastreada sem necessidade e entrega à view um modelo próprio para edição. A consequência prática é uma página de edição leve, com os dados atuais e o token necessário para detectar conflitos.

A gravação da edição combina validação, transação explícita, renovação do token e tratamento de concorrência. No mesmo arquivo `Controllers/PetsController.cs`, a action `POST` trabalha com o valor original de `RowVersion`, controla a transação pelo Entity Framework Core e trata `DbUpdateConcurrencyException` com uma mensagem amigável.

```
[HttpPost]  
[ValidateAntiForgeryToken]  
public async Task<IActionResult> Edit(int id, PetEditViewModel viewModel)  
{  
    if (id != viewModel.Id)  
    {  
        return NotFound();  
    }  
  
    if (!ModelState.IsValid)  
    {  
        return View(viewModel);  
    }  
  
    var pet = await _dbContext.Pets.FirstOrDefaultAsync(pet => pet.Id == id);  
  
    if (pet is null)  
    {  
        return NotFound();  
    }  
  
    var rowVersionOriginal = Convert.FromBase64String(viewModel.RowVersionBase64);  
  
    _dbContext.Entry(pet)  
        .Property(item => item.RowVersion)  
        .OriginalValue = rowVersionOriginal;  
  
    pet.Nome = viewModel.Nome.Trim();  
    pet.Cidade = viewModel.Cidade.Trim();  
    pet.Descricao = viewModel.Descricao.Trim();  
    pet.Disponível = viewModel.Disponível;  
    pet.RowVersion = Guid.NewGuid().ToByteArray();  
  
    await using var transaction = await _dbContext.Database.  
BeginTransactionAsync();
```

```

try
{
    await _dbContext.SaveChangesAsync();
    await transaction.CommitAsync();

    TempData["MensagemSucesso"] = $"Cadastro de {pet.Nome} atualizado com
sucesso.";

    return RedirectToAction(nameof(Index));
}
catch (DbUpdateConcurrencyException)
{
    await transaction.RollbackAsync();

    ModelState.AddModelError(
        string.Empty,
        "Este cadastro foi alterado por outra operação. Revise os dados
atuais antes de salvar novamente.");

    var petAtual = await _dbContext.Pets
        .AsNoTracking()
        .FirstOrDefaultAsync(item => item.Id == id);

    if (petAtual is not null)
    {
        viewModel.RowVersionBase64 =
Convert.ToBase64String(petAtual.RowVersion);
    }

    return View(viewModel);
}
}

```

O valor original de `RowVersion` é informado ao EF Core para que ele possa comparar a versão lida com a versão atual do banco. Antes da gravação, a aplicação renova o token. A transação criada por `BeginTransactionAsync` representa a unidade de trabalho; em caso de sucesso, ela é confirmada, e em caso de conflito, é desfeita. O objeto retornado pelo EF Core encapsula a transação do provedor, associada ao conceito de `DbTransaction`.

Para que o conflito possa ser reproduzido pela própria interface, o controller também recebeu a action `SimulateConcurrency`. Ela renova o token de um pet sem passar pelo formulário original, criando uma alteração concorrente didática.

```

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> SimulateConcurrency(int id)
{
    var pet = await _dbContext.Pets.FirstOrDefaultAsync(pet => pet.Id == id);

```

```
if (pet is null)
{
    return NotFound();
}
pet.RowVersion = Guid.NewGuid().ToByteArray();

await _dbContext.SaveChangesAsync();

TempData["MensagemSucesso"] = "A versão do cadastro foi renovada para
simular uma atualização concorrente.";

return RedirectToAction(nameof(Edit), new { id });
}
```

Essa action não representa uma função de negócio final do PetMatch e não deveria permanecer em uma aplicação publicada para usuários reais. Ela existe apenas para tornar visível o comportamento de concorrência durante o estudo. O leitor consegue abrir uma edição, acionar a simulação em outra aba e tentar salvar a aba original. O resultado esperado é a mensagem de conflito na view, sem perda silenciosa de dados.

A view `Views/Pets/Edit.cshtml`, no projeto `PetMatch.Web`, exibe o formulário de edição, envia o token em campo oculto e oferece o botão de simulação. O trecho a seguir mostra os elementos centrais da página.

```
@model PetEditViewModel

@{
    ViewData["Title"] = "Editar cadastro";
}

<partial name="_MvcHero" />

<section class="form-card">
    <form asp-action="Edit" asp-route-id="@Model.Id" method="post"
class="pet-form">
        <input asp-for="Id" type="hidden" />
        <input asp-for="RowVersionBase64" type="hidden" />

        <div asp-validation-summary="All" class="validation-summary"></div>

        <label asp-for="Nome"></label>
        <input asp-for="Nome" />
        <span asp-validation-for="Nome" class="field-validation"></span>

        <label asp-for="Cidade"></label>
        <input asp-for="Cidade" />
        <span asp-validation-for="Cidade" class="field-validation"></span>

        <label asp-for="Descricao"></label>
        <textarea asp-for="Descricao" rows="4"></textarea>
        <span asp-validation-for="Descricao" class="field-validation"></span>
    </form>
</section>
```

```

<label class="check-row">
  <input asp-for="Disponivel" />
  Disponível para adoção
</label>

<div class="form-actions">
  <a asp-controller="Pets" asp-action="Index" class="secondary-action"
&gtVoltar</a>
  <button type="submit">Salvar alterações</button>
</div>
</form>

<form asp-action="SimulateConcurrency" asp-route-id="@Model.Id"
method="post" class="simulation-box">
  <p>Use esta ação para simular outra operação alterando o mesmo
cadastro.</p>
  <button type="submit">Simular atualização concorrente</button>
</form>
</section>

```

A view evidencia como a concorrência chega à interface. O usuário não vê bytes nem detalhes internos do EF Core, mas o formulário transporta `RowVersionBase64` em campo oculto. O resumo de validação também recebe a mensagem de conflito quando a gravação falha por versão desatualizada. Assim, um problema técnico de consistência aparece como feedback compreensível.

A listagem também foi ajustada para conduzir o usuário ao fluxo de edição. Em `Views/Shared/_PetCard.cshtml`, cada card passou a exibir o link Editar cadastro, usando Tag Helpers para gerar a rota correta.

```

<a class="secondary-action"
  asp-controller="Pets"
  asp-action="Edit"
  asp-route-id="@Model.Id">
  Editar cadastro
</a>

```

Esse link conecta a experiência visual ao novo fluxo de persistência. A página `/Pets` continua mostrando cards vindos do banco, mas agora cada card permite revisar e alterar o cadastro com controle de concorrência.

A eficiência das leituras também foi aprimorada nos endpoints. No arquivo `Endpoints/PetEndpoints.cs`, a API continua nos mesmos caminhos públicos, mas passa a consultar com `AsNoTracking`, aplicar filtros antes da materialização e projetar `PetResponse`, sem expor `RowVersion`.

```

var consulta = dbContext.Pets
  .AsNoTracking()
  .Where(pet => pet.Disponivel);

```

```
if (!string.IsNullOrWhiteSpace(especie))
{
    consulta = consulta.Where(pet => pet.Especie == especie);
}
var pets = await consulta
    .OrderBy(pet => pet.Nome)
    .Select(pet => new PetResponse(
        pet.Id,
        pet.Nome,
        pet.Especie,
        pet.Porte,
        pet.IdadeAproximada,
        pet.Cidade,
        pet.Descricao,
        pet.Icone,
        pet.Disponivel,
        pet.Caracteristicas))
    .ToListAsync();

return Results.Ok(pets);
```

O `AsNoTracking` evita rastreamento desnecessário em consultas somente de leitura. Os filtros são aplicados antes de `ToListAsync`, permitindo que o banco participe da seleção dos dados. A projeção para `PetResponse` evita que detalhes internos da entidade, como `RowVersion`, sejam publicados pela API.

O resultado visual esperado é uma área MVC que continua agradável e funcional. Em `/Pets`, os cards exibem o link `Editar cadastro` (figura 19).

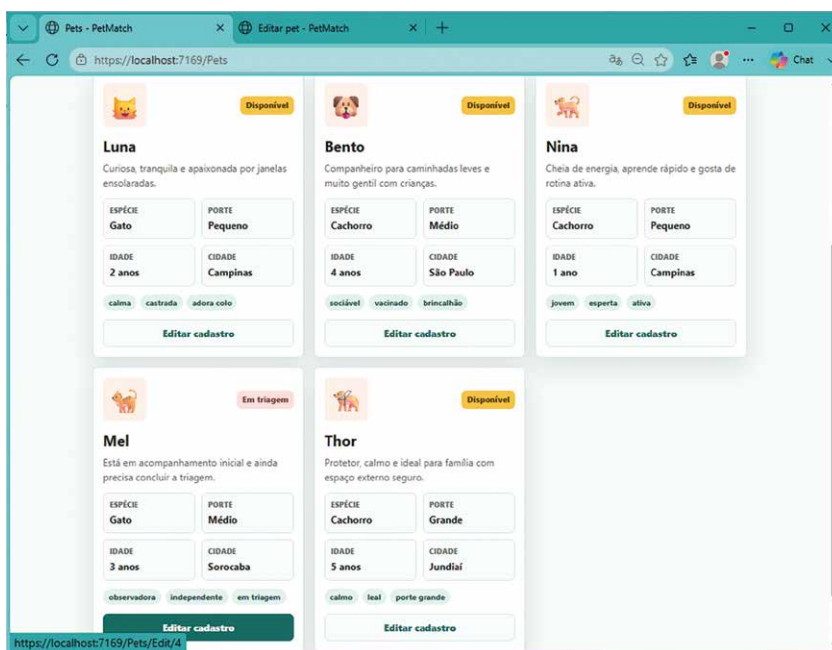


Figura 19 – Em `/Pets`, os cards exibem o link `Editar cadastro`. Note que ao passar o mouse sobre o botão (em verde) aparece o endereço `/Pets/Edit/4` (4 é o id da gatinha Mel)

Em `/Pets/Edit/{id}`, o usuário vê um formulário estilizado, com campos de edição, botão para simular concorrência e mensagens claras. Uma edição válida retorna à listagem com mensagem de sucesso. Uma tentativa de salvar com token antigo exibe conflito sem quebrar a aplicação. A API permanece disponível em `/api/pets`, agora com leitura eficiente e sem expor o token de concorrência (figuras 20 e 21).

PM PetMatch

Pets Cadastrar Página inicial

ÁREA MVC DO PETMATCH

Editar Mel

Atualize os dados do pet com controle de concorrência otimista.

Alteração concorrente detectada. O formulário agora mostra os dados mais recentes do banco.

EDIÇÃO SEGURA

Dados persistidos

O PetMatch usa a versão lida no início da edição para detectar alterações feitas por outra operação.

CENÁRIO DIDÁTICO

Simular atualização concorrente

Este pet foi alterado por outra operação. Os dados atuais foram recarregados: revise antes de salvar novamente.

Nome: Mel Espécie: Gato

Porte: Médio Idade aproximada: 3

Cidade: Sorocaba Ícone: Gato jovem

Descrição: Está em acompanhamento inicial e ainda precisa concluir a triagem.

Características: observadora, independente, em triagem

Figura 20 – Simulação de alteração concorrente na gatinha Mel (id=4). Note a mensagem de erro

PM PetMatch

Pets Cadastrar Página inicial

ÁREA MVC DO PETMATCH

Editar Mel

Atualize os dados do pet com controle de concorrência otimista.

A versão do pet foi renovada em uma operação separada. Volte à aba original e tente salvar para observar o conflito.

EDIÇÃO SEGURA

Dados persistidos

O PetMatch usa a versão lida no início da edição para detectar alterações feitas por outra operação.

CENÁRIO DIDÁTICO

Simular atualização concorrente

Nome: Mel Espécie: Gato

Porte: Médio Idade aproximada: 3

Cidade: Sorocaba Ícone: Gato jovem

Descrição: Está em acompanhamento inicial e ainda precisa concluir a triagem.

Características: observadora, independente, em triagem

☐ Disponível para adoção

Salvar alterações Voltar para listagem

Figura 21 – Simulação de alteração concorrente na gatinha Mel (id=4)

A inspeção do arquivo `petmatch.db` por uma ferramenta externa é opcional. A aplicação já comprova a persistência quando o pet cadastrado continua aparecendo após parar e iniciar novamente o projeto. Ainda assim, visualizar o banco pode ajudar o leitor a perceber que os dados estão realmente gravados em uma tabela SQLite. Uma opção dentro do Visual Studio é instalar a extensão SQLite and SQL Server Compact Toolbox.



Saiba mais

Para o SQLite, uma referência bibliográfica adequada é:

ALLEN, G.; OWENS, M. *The definitive guide to SQLite*. 2. ed. Berkeley: Apress, 2010.

Já para o SQLite and SQL Server Compact Toolbox, uma extensão/suíte de ferramentas para Visual Studio, SSMS e outros usos, desenvolvida e mantida por Erik Ejlskov Jensen, a referência mais indicada seria:

JENSEN, E. E. SQLite and SQL Server Compact Toolbox. *GitHub*, [s.d.]. Disponível em: <https://tinyurl.com/2nkn48ys>. Acesso em: 8 jul. 2026.

Para instalar a extensão (figura 22), acesse o menu `Extensions`, escolha `Manage Extensions...`, abra a aba de busca e pesquise por `SQLite and SQL Server Compact Toolbox`. Depois da instalação, reinicie o Visual Studio, se solicitado.

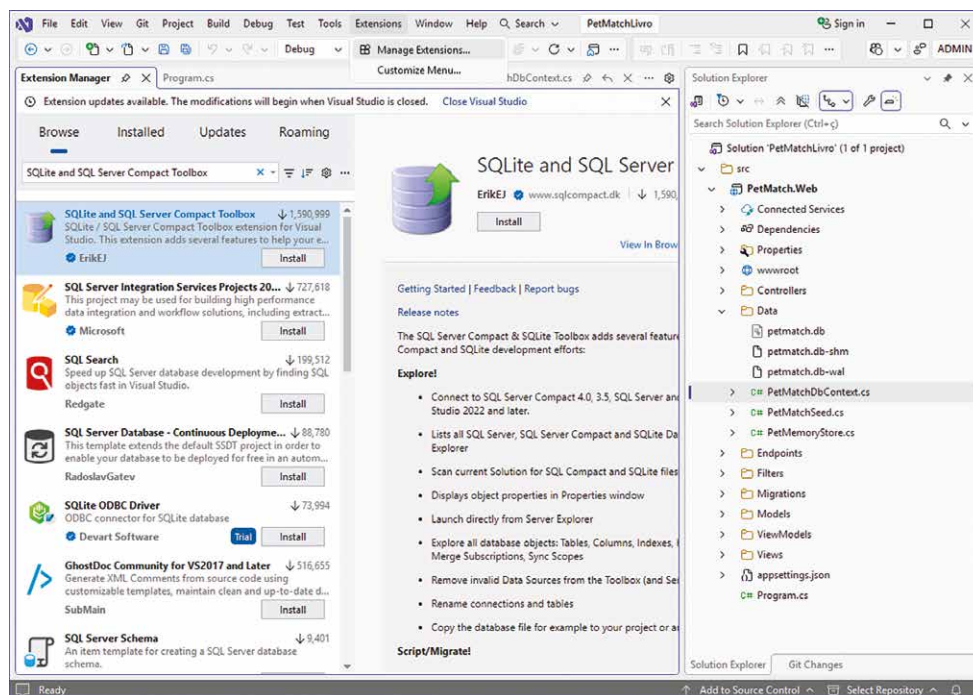


Figura 22 – Integrando o SQLite and SQL Server Compact Toolbox à IDE

A ferramenta aparecerá no menu **Tools** e funcionará como um explorador de bancos SQLite dentro da IDE (figura 23). Ao abrir o toolbox, use a opção de adicionar uma conexão SQLite e selecione o arquivo `petmatch.db` criado pela aplicação. Em seguida, expanda o banco, abra **Tables**, localize a tabela `Pets`, clique com o botão direito sobre ela e escolha **Edit Top 200 Rows**. Essa opção permite visualizar as primeiras linhas da tabela; se houver menos de 200 registros, todos os registros disponíveis serão exibidos.

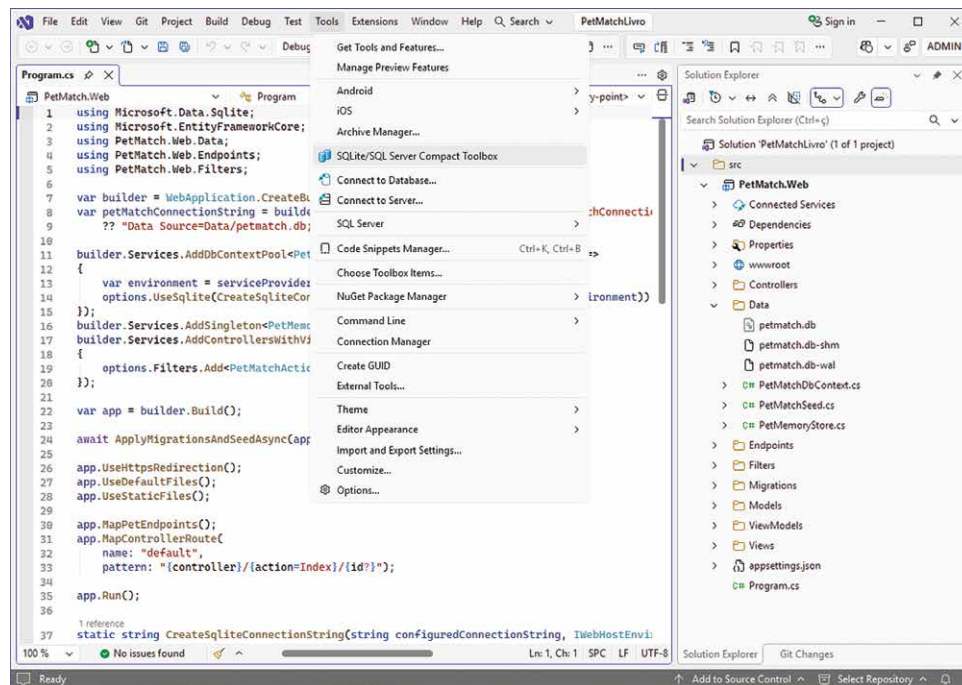


Figura 23 – SQLite/SQL Server Compact Toolbox nas opções do menu Tools

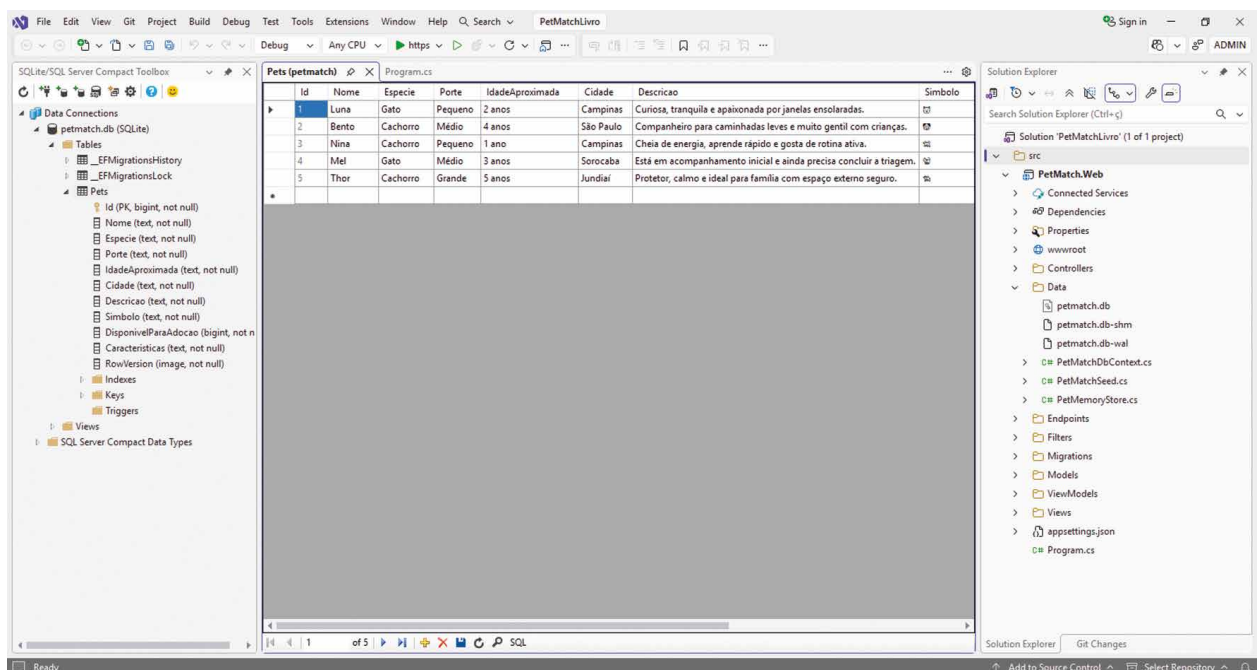


Figura 24 – Utilização do SQLite and SQL Server Compact Toolbox para visualização dos dados de petmatch.db

Depois de clicar em Edit Top 200 Rows, o Visual Studio abre uma grade no centro da tela (figura 24, ao centro). Essa grade se parece com uma planilha. Cada linha representa um pet salvo no banco e cada coluna representa uma informação desse pet, como Id, Nome, Espécie, Porte, IdadeAproximada, Cidade, Descrição e Símbolo.

4 APIS REST E OPENAPI

Uma aplicação web pode ser construída para apresentar páginas diretamente ao usuário, mas também pode ser projetada para disponibilizar funcionalidades e dados a outros sistemas. Nesse segundo caso, a interação não ocorre necessariamente por meio de uma interface visual renderizada no servidor, e sim por meio de contratos HTTP que permitem a comunicação entre aplicações distintas. APIs REST (Representational State Transfer) ocupam esse espaço arquitetural ao organizar recursos, operações, representações e respostas de maneira previsível sobre o protocolo HTTP. Elas permitem que uma aplicação exponha capacidades de seu domínio para navegadores, aplicativos móveis, serviços externos, painéis administrativos, integrações institucionais ou outras plataformas que precisem consumir informações de forma estruturada.

No desenvolvimento web contemporâneo, essa separação entre interface visual e interface de integração tornou-se cada vez mais importante. Um mesmo sistema pode atender usuários por páginas MVC e Razor, ao mesmo tempo em que fornece dados para um aplicativo móvel ou um serviço parceiro. A API passa a funcionar como uma fronteira pública ou interna da aplicação, definindo quais recursos podem ser acessados, quais operações são permitidas, quais dados devem ser enviados e quais respostas podem ser esperadas. Essa fronteira precisa ser clara porque, diferentemente de uma página voltada à interação humana, uma API é consumida por código. Pequenas ambiguidades em nomes, formatos, status de resposta ou regras de entrada podem dificultar integrações e produzir falhas difíceis de diagnosticar.

O REST propõe uma forma de pensar sistemas distribuídos em torno de recursos identificáveis e representações transferidas entre cliente e servidor. Em termos práticos, isso significa que a API não deve ser entendida apenas como um conjunto de métodos remotos chamados pela rede. Ela organiza o domínio em recursos acessíveis por endereços, utiliza verbos HTTP para indicar a intenção da operação e devolve respostas com códigos de status, cabeçalhos e corpos de mensagem adequados. Essa estrutura aproveita a semântica do próprio protocolo HTTP, tornando a comunicação mais expressiva e mais alinhada à natureza da web.

Em uma aplicação como o PetMatch, uma API REST pode representar uma forma de abrir o sistema para diferentes experiências de uso. Informações sobre pets disponíveis para adoção, detalhes de um animal, registros de interesse, alterações de disponibilidade ou consultas filtradas por espécie, porte e localização poderiam ser expostas como recursos consumíveis por outros clientes. Um aplicativo móvel voltado a adotantes, um painel de acompanhamento usado por uma organização protetora ou uma integração com uma plataforma municipal de bem-estar animal poderiam depender de uma API para acessar informações do PetMatch sem reproduzir diretamente suas páginas visuais. A API, nesse contexto, não substitui a aplicação orientada a páginas, mas amplia sua capacidade de integração.

A noção de contrato é central para compreender APIs REST. Quando uma aplicação disponibiliza um endpoint, ela estabelece uma promessa técnica para quem o consome. Essa promessa envolve o endereço do recurso, o método HTTP esperado, os parâmetros aceitos, o formato do corpo da requisição, as validações aplicadas, os códigos de resposta possíveis e a estrutura dos dados retornados. Um contrato bem definido reduz incertezas, facilita testes e permite que diferentes equipes ou sistemas evoluam com menor acoplamento. Em um cenário de adoção responsável, no qual informações sensíveis ou operacionalmente importantes podem circular entre plataformas, a clareza desse contrato contribui para a confiabilidade do ecossistema.

Os métodos HTTP desempenham papel fundamental nessa organização. Uma consulta deve ser distinguida de uma criação, uma atualização ou uma remoção, não apenas pelo nome do endereço, mas pela intenção semântica expressa na requisição. Essa distinção ajuda a tornar a API compreensível, previsível e compatível com mecanismos próprios da web, como cache, idempotência e tratamento padronizado de respostas. Da mesma forma, os códigos de status HTTP comunicam o resultado da operação de maneira objetiva. Uma resposta bem projetada informa se a solicitação foi bem-sucedida, se houve erro de validação, se o recurso não foi encontrado, se o usuário não tem autorização ou se ocorreu uma falha no servidor. Assim, a API conversa com seus consumidores por meio de uma linguagem técnica compartilhada.

A representação dos dados também exige cuidado. APIs REST frequentemente utilizam JSON como formato de troca, por ser leve, legível e amplamente aceito por diferentes plataformas e linguagens. Entretanto, o formato em si não resolve todos os problemas de integração. É necessário decidir quais propriedades serão expostas, como nomes serão padronizados, quais dados internos devem permanecer ocultos, como erros serão descritos e como mudanças futuras serão tratadas sem quebrar os clientes existentes. No PetMatch, por exemplo, a representação pública de um pet para adoção pode não coincidir integralmente com o modelo interno usado pela aplicação. A API deve expor aquilo que é necessário ao consumidor, preservando a coerência do domínio e evitando vazamento de detalhes de implementação.

À medida que uma API cresce, sua documentação deixa de ser acessória e torna-se parte indispensável do contrato. Consumidores precisam saber como chamar os endpoints, quais parâmetros informar, que formato enviar, que respostas esperar e como interpretar falhas. Documentação escrita manualmente, quando desconectada do código, tende a se desatualizar com facilidade. A OpenAPI responde a esse problema ao oferecer uma especificação padronizada para descrever APIs HTTP de maneira estruturada, legível por pessoas e processável por ferramentas. Com ela, a API pode ser documentada, explorada e validada a partir de um contrato formal.

A OpenAPI permite representar endpoints, métodos, parâmetros, corpos de requisição, esquemas de resposta, códigos de status e informações gerais da API em um formato padronizado. Essa descrição pode alimentar interfaces interativas de exploração, nas quais desenvolvedores visualizam operações disponíveis e experimentam chamadas controladas. Também pode servir como base para geração de clientes em diferentes linguagens, facilitando o consumo da API por aplicações externas. Dessa forma, a documentação é um artefato técnico que participa do ciclo de desenvolvimento, teste, integração e manutenção.

No PetMatch, a presença de uma documentação OpenAPI ajudar a tornar explícitas as possibilidades de integração do sistema. Uma organização parceira poderia compreender quais dados estão disponíveis, como consultar pets por critérios específicos, como enviar uma manifestação de interesse ou como interpretar uma resposta de validação. Um desenvolvedor responsável por um aplicativo móvel poderia gerar parte do cliente de comunicação a partir do contrato documentado, reduzindo erros de digitação, divergências de formato e interpretações ambíguas. A especificação passa a atuar como ponte entre quem oferece a API e quem a consome.

O estudo de APIs REST e OpenAPI exige, portanto, uma mudança de perspectiva. Em tópicos voltados a páginas, o foco recai principalmente sobre a experiência visual do usuário no navegador. Ao tratar de APIs, o foco se desloca para a experiência de integração entre sistemas. A aplicação continua recebendo requisições HTTP e produzindo respostas, mas agora essas respostas precisam ser pensadas para consumidores programáticos. Isso exige precisão, estabilidade, clareza semântica e atenção aos contratos publicados. Uma API mal definida pode funcionar em testes locais, mas se tornar difícil de consumir, documentar e evoluir quando passa a atender diferentes clientes.



Saiba mais

Consulte no endereço a seguir a especificação oficial do OpenAPI, mantida pela OpenAPI Initiative. Ela define um padrão independente de linguagem para descrever APIs HTTP:

OPENAPI INITIATIVE. *OpenAPI Specification: version 3.2.0*: OpenAPI Initiative, 2025. Disponível em: <https://tinyurl.com/4nnwm5pj>. Acesso em: 7 jul. 2026.

Este tópico introduz APIs REST como uma forma organizada de transformar funcionalidades do domínio em contratos HTTP de integração. Também apresenta a OpenAPI como instrumento para documentar, explorar e reutilizar esses contratos de modo sistemático. No percurso evolutivo do PetMatch, essa abordagem amplia o alcance da aplicação, permitindo que suas informações e operações possam ser consumidas por outras interfaces e serviços sem abandonar os princípios de clareza, responsabilidade e consistência que orientam o sistema. Compreender essa camada é essencial para desenvolver aplicações .NET capazes não apenas de servir páginas, mas de participar de ecossistemas distribuídos de software.

4.1 Recursos, paginação/ordenação/filtros e erros (Problem Details)

Uma API HTTP se torna mais previsível quando seus endereços representam recursos claros e suas respostas seguem contratos estáveis. No PetMatch, a coleção de pets disponível em `/api/pets` representa um recurso de listagem, enquanto `/api/pets/{id}` representa um pet específico. Essa organização permite que o cliente da API compreenda a intenção da chamada pelo próprio endereço e pelo método HTTP usado. A preocupação central não é apenas devolver JSON, mas oferecer uma superfície de comunicação consistente, em que sucesso, ausência de dados, parâmetros inválidos e erros sejam tratados de forma compreensível.

O conceito de recurso ajuda a separar a API de detalhes internos da aplicação. A entidade persistente `Pet` possui propriedades necessárias ao EF Core, inclusive dados que não devem ser publicados, como o token de concorrência `RowVersion`. Já a resposta HTTP deve expor apenas o que o cliente precisa conhecer. Por isso, o PetMatch passa a usar contratos próprios em `Contracts/PetResponse.cs`, `Contracts/PetQueryParameters.cs` e `Contracts/PagedResponse.cs`. Esses arquivos criam uma fronteira mais nítida entre o modelo persistido e o formato público da API.

A paginação protege a aplicação e o contrato HTTP. Mesmo que o PetMatch ainda tenha poucos registros, a API não deve depender da suposição de que toda listagem será pequena. O endpoint `/api/pets` passa a aceitar `page` e `pageSize`, devolvendo apenas uma parte da coleção e informando metadados sobre a resposta. Esse formato permite que uma interface navegue pelos resultados com clareza e evita que a API precise enviar uma lista indefinidamente crescente em uma única resposta.

No projeto `PetMatch.Web`, o arquivo `Contracts/PagedResponse.cs` define o envelope usado nas respostas paginadas. Ele agrupa os itens retornados e os metadados necessários para que o cliente entenda a posição da página dentro do conjunto total.

```
namespace PetMatch.Web.Contracts;

public sealed record PagedResponse<T>(
    IReadOnlyList<T> Items,
    int Page,
    int PageSize,
    int TotalItems,
    int TotalPages);
```

Esse contrato demonstra que a resposta da listagem deixa de ser apenas um array. O cliente recebe os pets da página atual e também informações para exibir navegação, como total de itens e total de páginas. Essa estrutura é útil para a nova página estática `/api.html`, que mostra os resultados e os metadados da consulta no navegador (figuras 25 e 26).

Os filtros e a ordenação refinam a consulta sobre a coleção de recursos. Os filtros reduzem o conjunto de pets retornados por critérios como espécie, porte, cidade e disponibilidade. A ordenação define a sequência da resposta, por nome, cidade ou idade aproximada. Esses parâmetros fazem parte do contrato da API, eles precisam ser previsíveis, ter nomes estáveis e produzir respostas coerentes sem expor a consulta interna do EF Core. A figura 26 mostra os filtros, bem como a ordenação (no exemplo da figura, por nome do pet). Também inserimos o botão "ver JSON" para mostrar o conteúdo JSON da pesquisa.

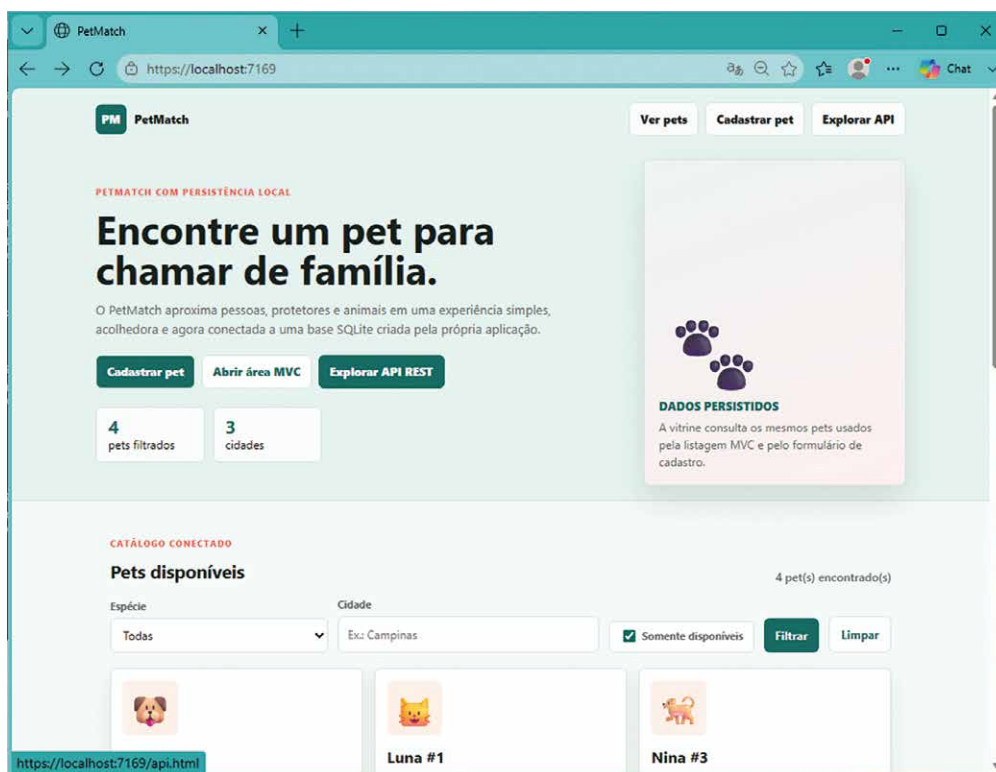


Figura 25 – PetMatch com o botão para testar a API REST

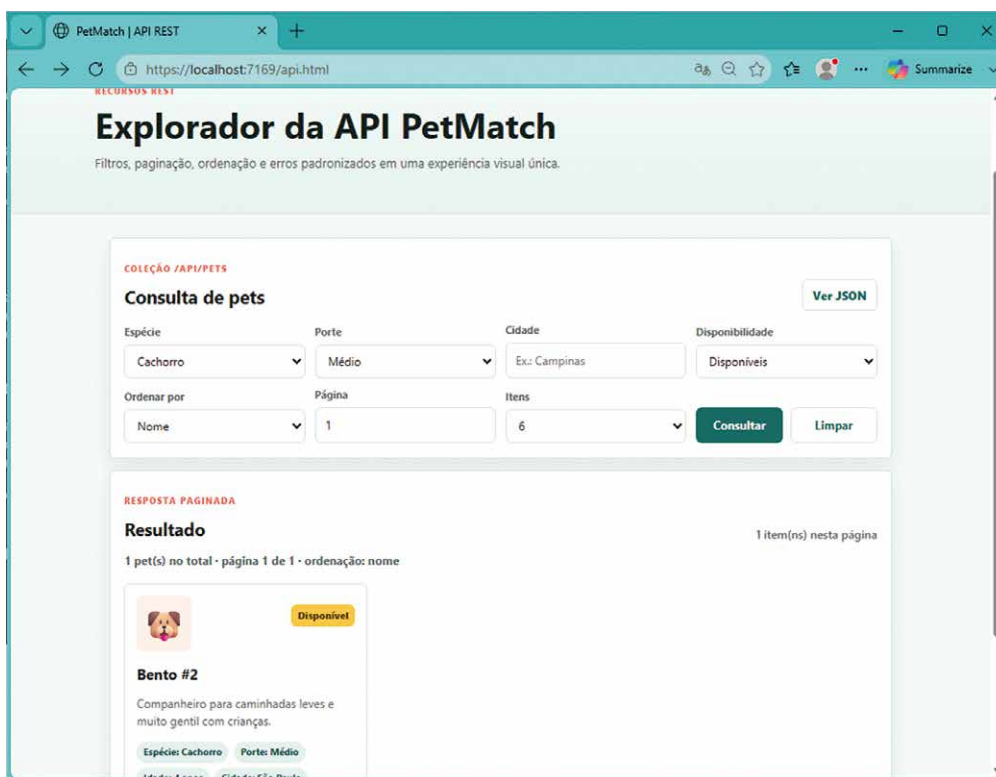


Figura 26 – Nova página estática /api.html. Note os filtros (no exemplo: Espécie "Cachorro" e Porte "Médio")

No projeto `PetMatch.Web`, o arquivo `Contracts/PetQueryParameters.cs` concentra os parâmetros aceitos por `/api/pets`. Ele também aplica limites didáticos ao tamanho da página, impedindo que uma chamada solicite uma quantidade exagerada de itens.

```
namespace PetMatch.Web.Contracts;

public sealed class PetQueryParameters
{
    private const int MaxPageSize = 20;

    public int Page { get; set; } = 1;

    private int _pageSize = 6;

    public int PageSize
    {
        get => _pageSize;
        set => _pageSize = Math.Clamp(value, 1, MaxPageSize);
    }

    public string? Espécie { get; set; }

    public string? Porte { get; set; }

    public string? Cidade { get; set; }

    public bool? Disponível { get; set; }

    public bool? DisponívelParaAdocao { get; set; }

    public string? Sort { get; set; }
}
```

Esse objeto representa a entrada da listagem. O parâmetro `DisponívelParaAdocao` foi mantido por compatibilidade com o filtro antigo da interface, enquanto `Disponível` expressa de modo mais direto o estado do recurso na API. A limitação de `PageSize` transforma uma regra de contrato em código explícito, evitando que a paginação seja apenas uma convenção informal.

A resposta de cada pet também foi formalizada. No projeto `PetMatch.Web`, o arquivo `Contracts/PetResponse.cs` define o DTO (Data Transfer Object) publicado pela API, sem expor `RowVersion` ou detalhes internos da entidade persistente.

```
namespace PetMatch.Web.Contracts;

public sealed record PetResponse(
    int Id,
    string Nome,
    string Espécie,
    string Porte,
    int IdadeAproximada,
    string Cidade,
    string Descricao,
```

```
string Icone,  
bool Disponivel,  
string[] Caracteristicas);
```

Esse contrato preserva os dados úteis para clientes da API e para a página explorável. Ao mesmo tempo, impede que a API publique informações que pertencem ao controle interno da persistência e da concorrência. A projeção para `PetResponse` também torna a consulta mais intencional, pois a aplicação escolhe o formato da resposta em vez de devolver a entidade diretamente.

No arquivo `Endpoints/PetEndpoints.cs`, o endpoint `GET /api/pets` passa a aplicar filtros, ordenação, paginação e projeção. A consulta usa `AsNoTracking`, pois a listagem é uma leitura sem alteração de estado e só materializa os resultados depois de aplicar os critérios recebidos.

```
api.MapGet("/pets", async (  
    PetMatchDbContext dbContext,  
    [AsParameters] PetQueryParameters query) =>  
{  
    var page = Math.Max(query.Page, 1);  
  
    var consulta = dbContext.Pets  
        .AsNoTracking()  
        .AsQueryable();  
    var disponivel = query.Disponivel ?? query.DisponivelParaAdocao;  
  
    if (disponivel.HasValue)  
    {  
        consulta = consulta.Where(pet => pet.Disponivel == disponivel.Value);  
    }  
  
    if (!string.IsNullOrEmpty(query.Especie))  
    {  
        consulta = consulta.Where(pet => pet.Especie == query.Especie);  
    }  
  
    if (!string.IsNullOrEmpty(query.Porte))  
    {  
        consulta = consulta.Where(pet => pet.Porte == query.Porte);  
    }  
  
    if (!string.IsNullOrEmpty(query.Cidade))  
    {  
        consulta = consulta.Where(pet => pet.Cidade.Contains(query.Cidade));  
    }  
  
    consulta = query.Sort?.ToLowerInvariant() switch  
    {  
        "cidade" => consulta.OrderBy(pet => pet.Cidade).ThenBy(pet => pet.Nome),  
        "idade" => consulta.OrderBy(pet => pet.IdadeAproximada).ThenBy(pet =>  
pet.Nome),  
        _ => consulta.OrderBy(pet => pet.Nome)  
    };  
}
```

```
var totalItems = await consulta.CountAsync();
var totalPages = (int)Math.Ceiling(totalItems / (double)query.PageSize);

var pets = await consulta
    .Skip((page - 1) * query.PageSize)
    .Take(query.PageSize)
    .Select(pet => new PetResponse(
        pet.Id,
        pet.Nome,
        pet.Especie,
        pet.Porte,
        pet.IdadeAproximada,
        pet.Cidade,
        pet.Descricao,
        pet.Icone,
        pet.Disponivel,
        pet.Caracteristicas))
    .ToListAsync();

return Results.Ok(new PagedResponse<PetResponse>(
    pets,
    page,
    query.PageSize,
    totalItems,
    totalPages));
```

Esse código demonstra um contrato mais maduro para listagem. Os filtros são aplicados antes da materialização, a ordenação é limitada a opções conhecidas, a paginação usa `Skip` e `Take` e a resposta publica um envelope padronizado. A API continua acessível sem query string, pois `page` e `pageSize` são opcionais, mas também passa a responder chamadas como `/api/pets?page=1&pageSize=2`.



Figura 27 – Resultado em `/api/pets?page=1&pageSize=2`

Os erros também fazem parte do contrato. Uma API que retorna formatos diferentes para cada falha obriga o cliente a tratar exceções de maneira frágil. O padrão `Problem Details` oferece um formato reconhecível para erro HTTP, com campos como `title`, `status` e `detail`. No `PetMatch`, esse formato passa a ser usado em falhas como pet inexistente e dados inválidos em manifestações de interesse.

No projeto `PetMatch.Web`, o suporte a `Problem Details` é registrado no arquivo `Program.cs`, junto aos demais serviços da aplicação e antes da chamada a `builder.Build()`:

```
builder.Services.AddProblemDetails();
```

Esse registro habilita os serviços relacionados à produção de respostas de erro padronizadas. Nos endpoints, o uso de `Results.Problem` permite definir o status HTTP e apresentar informações compreensíveis sobre a falha. O endpoint de consulta por identificador, localizado em `Endpoints/PetEndpoints.cs`, preserva o caminho `/api/pets/{id:int}`, mas passa a responder com `Problem Details` quando o pet solicitado não existe.

```
api.MapGet("/pets/{id:int}", async (PetMatchDbContext dbContext, int id) =>
{
    var pet = await dbContext.Pets
        .AsNoTracking()
        .Where(pet => pet.Id == id)
        .Select(pet => new PetResponse(
            pet.Id,
            pet.Nome,
            pet.Especie,
            pet.Porte,
            pet.IdadeAproximada,
            pet.Cidade,
            pet.Descricao,
            pet.Icone,
            pet.Disponivel,
            pet.Caracteristicas))
        .FirstOrDefaultAsync();
    return pet is null
        ? Results.Problem(
            title: "Pet não encontrado.",
            detail: $"Não existe pet cadastrado com o identificador {id}.",
            statusCode: StatusCodes.Status404NotFound)
        : Results.Ok(pet);
});
```

Esse trecho mostra que a ausência de um recurso não é uma mensagem improvisada. O cliente recebe um status HTTP adequado e um corpo padronizado. A resposta continua sendo legível no navegador, mas também se torna mais fácil de tratar por uma aplicação consumidora. Criamos o botão "Simular erro" para facilitar a visualização desse conceito. A figura 28 ilustra uma situação de simulação.

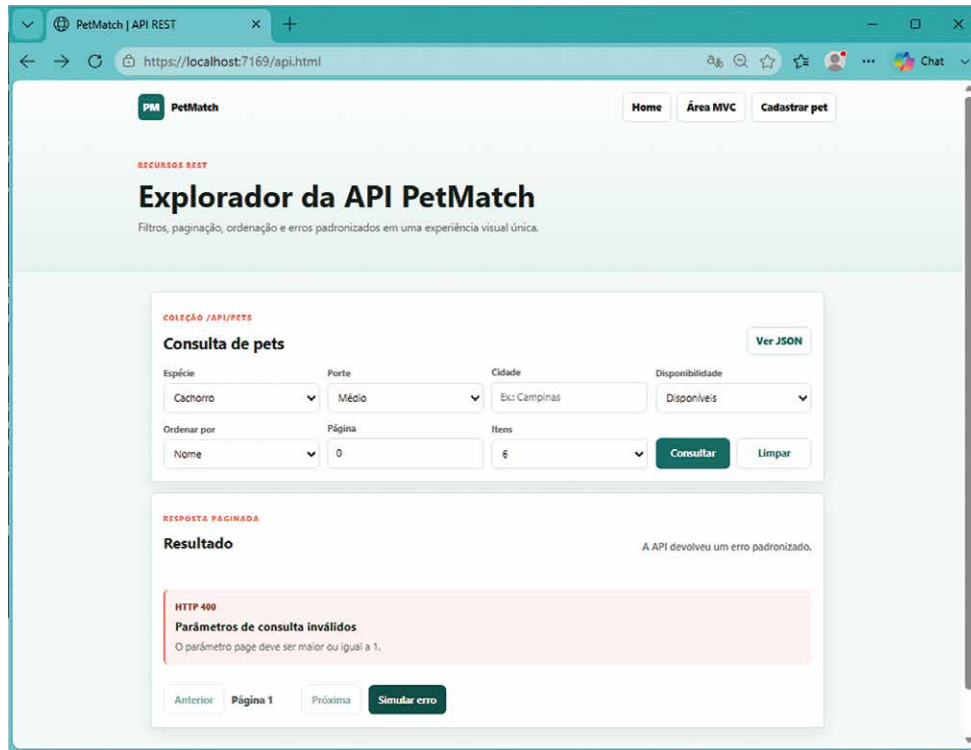


Figura 28 – Simulação de erro na página com o código de erro e a mensagem ao usuário

Para tornar a API explorável sem ferramentas externas, criamos no projeto a página estática `wwwroot/api.html` e o script `wwwroot/js/petmatch-api.js`. Essa página contém controles para espécie, porte, cidade, disponibilidade, ordenação, página e tamanho da página. O JavaScript monta a query string, chama `/api/pets`, exibe os cards retornados e mostra um painel amigável quando a resposta contém `Problem Details`.

```
async function carregarPets() {
    const parametros = new URLSearchParams(new FormData(formulario));
    const resposta = await fetch(`/api/pets?${parametros.toString()}`);
    const conteudo = await resposta.json();

    if (!resposta.ok) {
        mostrarProblemDetails(conteudo);
        return;
    }

    renderizarMetadados(conteudo);
    renderizarPets(conteudo.items);
}
```

Esse trecho resume o papel da página explorável. Ela não substitui documentação formal da API, mas permite experimentar o contrato pelo navegador: mudar filtros, observar metadados, navegar por páginas e visualizar erros padronizados. O arquivo `wwwroot/css/petmatch.css` foi ajustado para dar acabamento visual a essa experiência e a home `wwwroot/index.html` recebeu um botão para explorar API REST.

O resultado visual esperado é uma home que mantém os cards e adiciona acesso claro à exploração da API. Em `/api.html`, o leitor encontra um formulário de filtros, controles de ordenação e paginação, cards de pets, metadados da resposta e painel de erro padronizado. Os endpoints `/api/pets`, `/api/pets?page=1&pageSize=2`, `/api/pets/1` e `/api/pets/99999` (figura 29) continuam acessíveis diretamente no navegador.

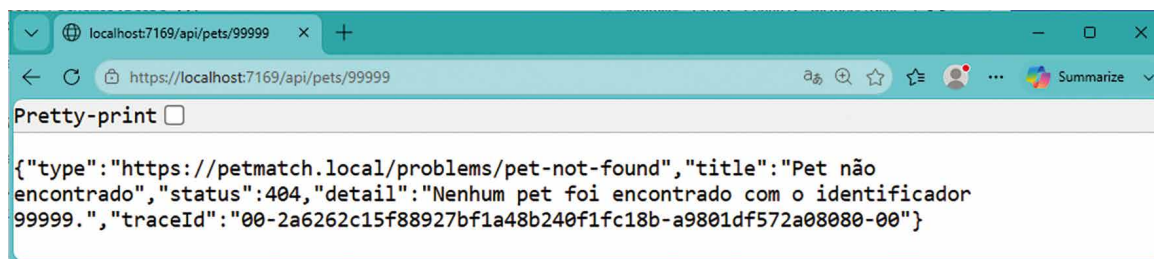


Figura 29 – Resultado JSON da requisição `/api/pets/99999`
(não existe pet com esse identificador na base de dados)

4.2 Documentação com Swagger e geração de cliente TypeScript (NSwag)

Uma API não é apenas um conjunto de rotas que respondem em JSON. Quando ela passa a ser consumida por uma interface, por outro serviço ou por uma equipe diferente daquela que a implementou, ela precisa de um contrato explícito. Esse contrato descreve quais recursos existem, quais parâmetros cada operação aceita, quais respostas podem ser produzidas, quais códigos HTTP indicam sucesso ou falha e qual é a forma dos dados trafegados. No ecossistema ASP.NET Core, OpenAPI cumpre esse papel, é uma especificação legível por ferramentas que transforma a API em um documento formal, capaz de orientar documentação, testes exploratórios, validação de integração e geração de clientes.

O Swagger é frequentemente usado como sinônimo de OpenAPI, mas convém distinguir os termos. OpenAPI é a especificação do contrato; Swagger é o conjunto de ferramentas e a interface interativa que permitem visualizar e experimentar esse contrato no navegador. A consequência prática é importante, a aplicação não deve depender da tela do Swagger para funcionar, nem transformar Swagger em parte da experiência final do usuário. O PetMatch continua sendo um sistema de adoção responsável de pets, com home, listagem, cadastro e edição. O Swagger entra como instrumento de desenvolvimento, restrito ao ambiente de desenvolvimento, sem botão ou link nas telas públicas da aplicação.

A geração de cliente TypeScript surge quando o contrato OpenAPI deixa de ser apenas documentação e passa a orientar código. Em uma integração manual, o front-end tende a reescrever URLs, nomes de parâmetros, tipos de resposta e formatos de erro. Essa duplicação é frágil: se a API muda e o cliente não acompanha, a divergência aparece em tempo de execução. Com uma ferramenta como NSwag, o contrato OpenAPI pode gerar um arquivo TypeScript com tipos e métodos correspondentes às operações descritas. A OpenAPI Generator é uma alternativa conhecida nesse mesmo espaço, mas o PetMatch usará a NSwag por integração direta ao fluxo do projeto via NuGet e MSBuild.

Neste tópico, duas ferramentas atuam em momentos diferentes. O Swagger, por meio do pacote `Swashbuckle.AspNetCore`, é responsável por expor a documentação interativa da API no navegador, em ambiente de desenvolvimento. É ele que permite abrir `/swagger`, consultar os endpoints, verificar parâmetros, visualizar schemas e testar requisições manualmente.

O NSwag entra em uma etapa posterior, não substitui a tela do Swagger nem aparece como uma página visual da aplicação. Sua função é ler o documento OpenAPI gerado pela API e produzir código TypeScript a partir dele. No PetMatch, essa configuração estará no arquivo `nswag.json`, que define o documento de entrada e o arquivo de saída do cliente TypeScript. Conforme detalharemos adiante, o resultado esperado é o arquivo `Clients/TypeScript/petmatch-api-client.ts`, contendo uma classe `PetMatchApiClient` com métodos e tipos derivados do contrato da API. Assim, Swagger e NSwag não fazem a mesma coisa. Swagger torna o contrato visível e testável no navegador. NSwag usa esse contrato como fonte para gerar código cliente.

Esse tópico se apoia nos contratos `PagedResponse<T>`, `PetQueryParameters`, `PetResponse` e `Problem Details`, já criados para a API. Esses elementos tornam a documentação mais útil porque a API já possui tipos explícitos para listagem paginada, filtros, ordenação e respostas de erro. Documentar endpoints que retornam objetos anônimos ou formatos variáveis produz um contrato pobre. Documentar endpoints que usam contratos nomeados, respostas padronizadas e metadados claros produz uma fonte confiável para ferramentas e para o front-end.

No PetMatch, a página pública `wwwroot/api.html`, criada anteriormente para explorar a API, foi removida, assim como `wwwroot/js/petmatch-api.js`. A home deixa de exibir chamada técnica para "Explorar API REST". A documentação passa para Swagger em desenvolvimento, enquanto a interface final permanece orientada ao usuário: ver pets, cadastrar pets e editar cadastros.

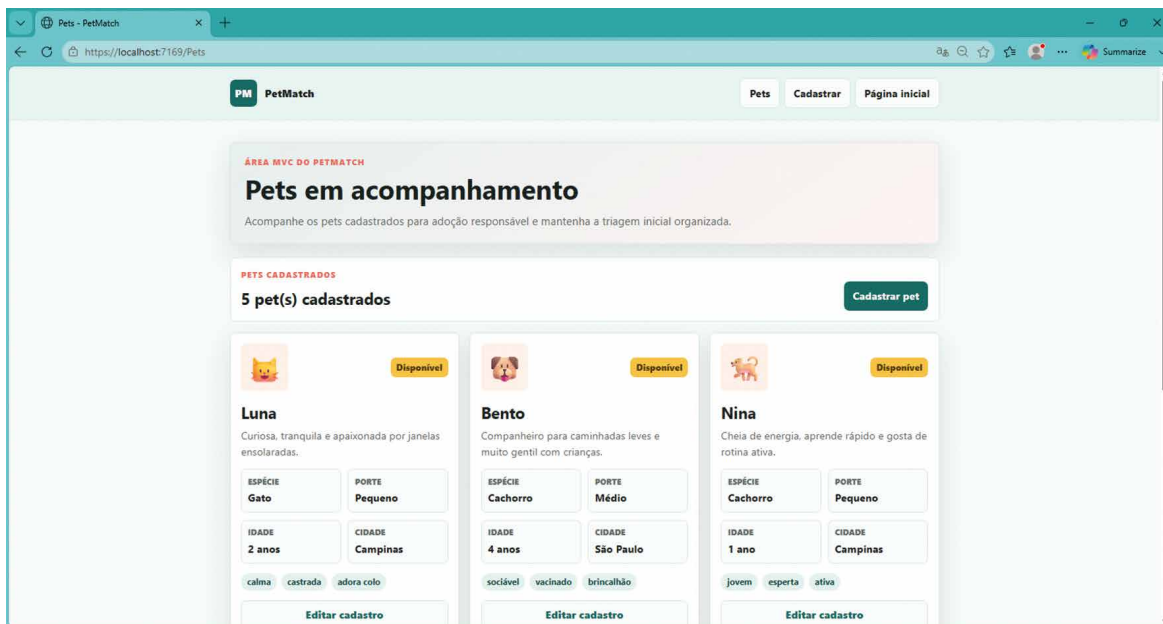


Figura 30 – Nova interface do PetMatch com aparência de produto voltado à adoção, não de painel de desenvolvimento

No projeto `PetMatch.Web`, a configuração de Swagger começa em `Program.cs`. O código registra o explorador de endpoints e o gerador Swagger, associando o documento à classe de metadados `PetMatchOpenApi`. A interface Swagger é habilitada apenas quando a aplicação está em ambiente de desenvolvimento.

```
builder.Services.AddEndpointsApiExplorer();

builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc(
        PetMatchOpenApi.DocumentName,
        PetMatchOpenApi.CreateInfo());
});
```

Esse trecho prepara a geração do documento OpenAPI. Ele não altera as rotas da aplicação, não cria tela pública para o usuário final e não substitui os contratos já implementados. Sua função é permitir que os endpoints Minimal API sejam descritos em um documento consultável por Swagger e por ferramentas de geração.

É importante distinguir dois momentos da documentação. Quando a aplicação está em execução, o Swagger expõe o contrato em `/swagger/v1/swagger.json` e apresenta a interface interativa em `/swagger`. Esse documento é produzido para consulta no navegador durante o desenvolvimento. Ele não deve ser confundido com um arquivo físico gerado durante a compilação do projeto. Essa diferença é essencial para entender onde o NSwag entra. A tela do Swagger permite visualizar e testar a API, mas não é ela que gera diretamente o cliente TypeScript. Para que o NSwag seja executado durante o build, o projeto precisa ter antes um documento OpenAPI gravado em disco. No PetMatch, esse arquivo será gerado em `obj/openapi/v1.json`. Somente depois disso o NSwag lê esse contrato e produz o arquivo TypeScript em `Clients/TypeScript/petmatch-api-client.ts`.

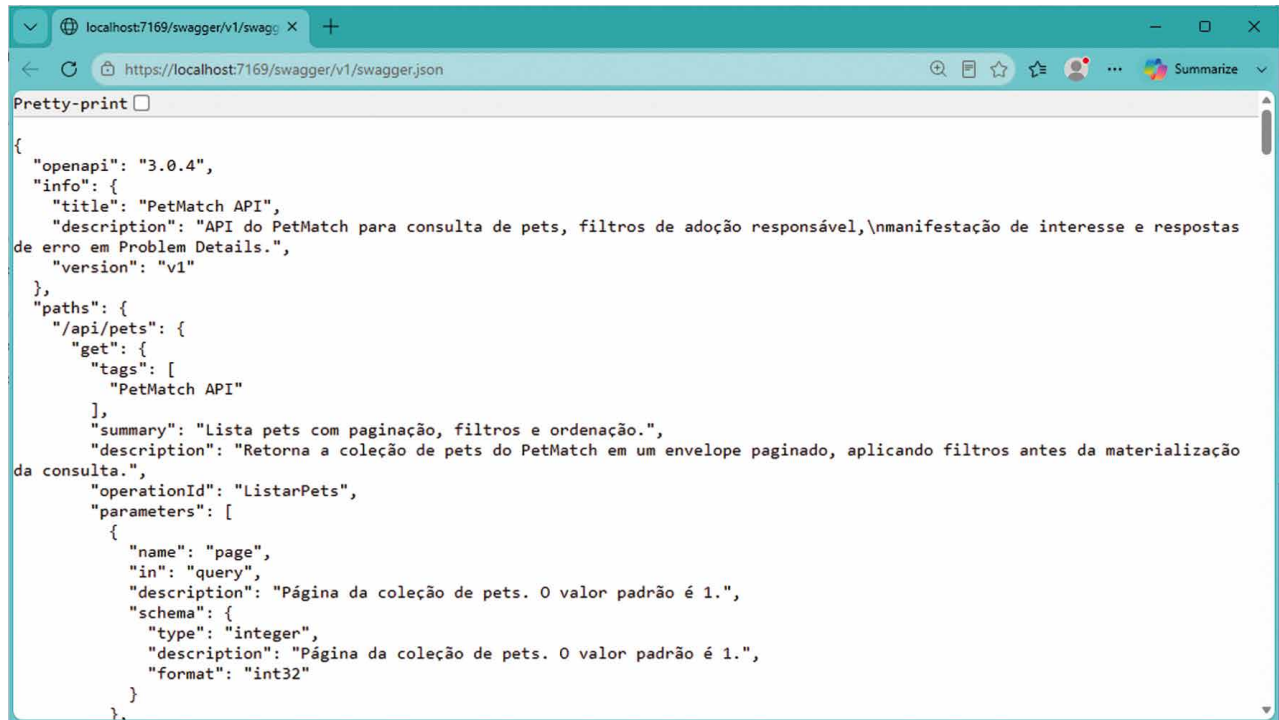
Ainda em `Program.cs`, o Swagger UI deve ser ativado de forma restrita. O trecho fica depois da construção da aplicação e antes do mapeamento final das rotas.

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();

    app.UseSwaggerUI(options =>
    {
        options.SwaggerEndpoint(
            "/swagger/v1/swagger.json",
            "PetMatch API v1");
    });
}
```

Esse código reforça a separação entre ferramenta de desenvolvimento e interface de usuário. Em desenvolvimento, o contrato pode ser aberto no navegador em `/swagger` (figura 32) e o JSON em `/swagger/v1/swagger.json` (figura 31). Nas telas do PetMatch, entretanto, não há chamada

para Swagger. O usuário continua navegando pelas páginas de adoção, enquanto o contrato técnico permanece disponível para quem desenvolve ou integra a API.



```
{
  "openapi": "3.0.4",
  "info": {
    "title": "PetMatch API",
    "description": "API do PetMatch para consulta de pets, filtros de adoção responsável,\nmanifestação de interesse e respostas de erro em Problem Details.",
    "version": "v1"
  },
  "paths": {
    "/api/pets": {
      "get": {
        "tags": [
          "PetMatch API"
        ],
        "summary": "Lista pets com paginação, filtros e ordenação.",
        "description": "Retorna a coleção de pets do PetMatch em um envelope paginado, aplicando filtros antes da materialização da consulta.",
        "operationId": "ListarPets",
        "parameters": [
          {
            "name": "page",
            "in": "query",
            "description": "Página da coleção de pets. O valor padrão é 1.",
            "schema": {
              "type": "integer",
              "description": "Página da coleção de pets. O valor padrão é 1.",
              "format": "int32"
            }
          }
        ]
      }
    }
  }
}
```

Figura 31 – Conteúdo JSON de /swagger/v1/swagger.json

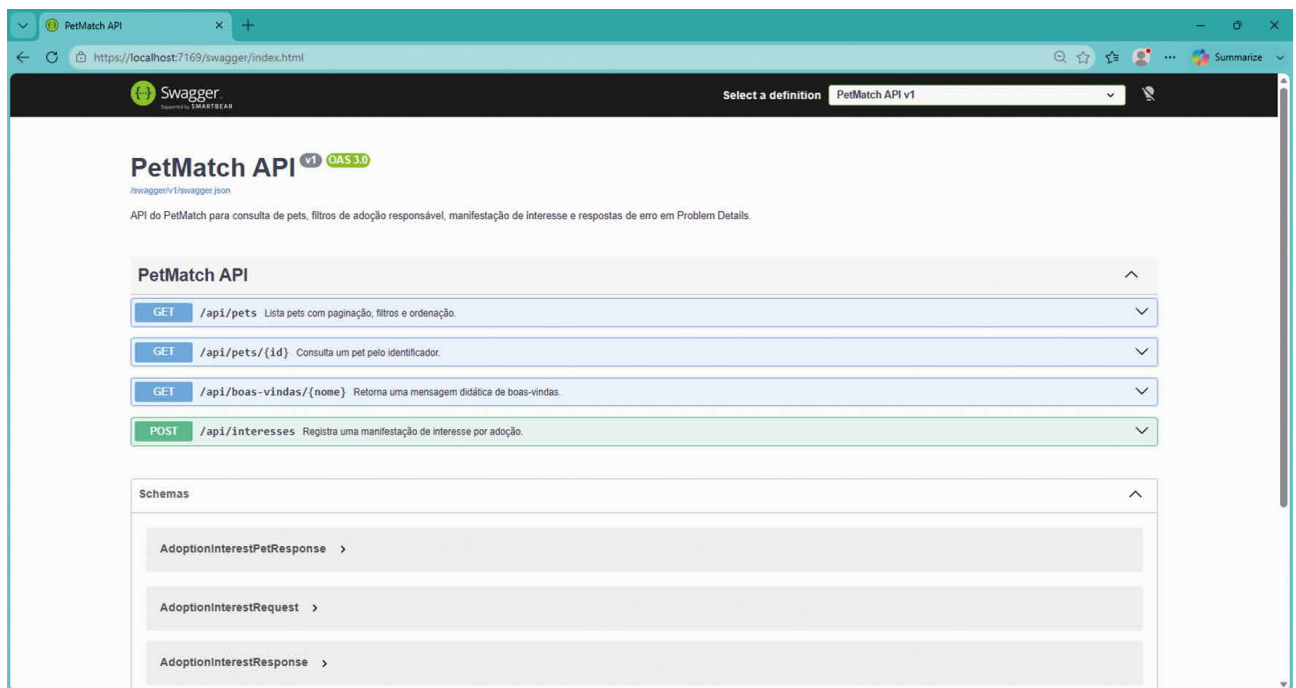


Figura 32 – Resultado da aplicação PetMatch em /swagger

A figura 32 mostra o Swagger UI da aplicação PetMatch API. Note que o Swagger funciona como uma página de documentação interativa da API. Em vez de o desenvolvedor precisar adivinhar quais endereços existem, quais métodos HTTP devem ser usados e quais dados cada rota espera, o Swagger apresenta essas informações de forma visual e organizada no navegador. O trecho `/swagger/index.html` é a página visual do Swagger. Ela não é uma página comum do sistema PetMatch para o usuário final, mas uma ferramenta de inspeção e teste da API.

Logo abaixo aparece o título PetMatch API, acompanhado da versão v1 e da indicação OAS 3.0. OAS significa OpenAPI Specification, isto é, uma especificação padronizada que descreve APIs HTTP. Em termos simples, essa marca informa que a API foi documentada seguindo o padrão OpenAPI, permitindo que ferramentas como o Swagger consigam exibir rotas, parâmetros, respostas e modelos de dados de forma organizada. Abaixo do título há um link para `/swagger/v1/swagger.json`. Esse arquivo é a descrição técnica da API em formato JSON. Ele contém os dados que o Swagger usa para montar essa interface visual. Um desenvolvedor pode abrir esse arquivo diretamente (figura 31), mas, para leitura humana, a página `swagger/index.html` é muito mais amigável. Note também que no topo aparece uma breve descrição da API com uma frase resume a finalidade do serviço. Ela informa que a API permite consultar pets, aplicar filtros, registrar interesse em adoção e devolver erros em um formato padronizado.

No centro da tela da figura 32 há um bloco chamado PetMatch API. Dentro dele aparecem as rotas disponíveis. Cada linha representa uma operação que a aplicação oferece. À esquerda de cada linha há um verbo HTTP, como `GET` ou `POST`. O verbo indica o tipo de ação que será feita. As linhas azuis marcadas com `GET` representam operações de consulta. Elas servem para buscar informações, sem criar ou alterar dados. Na tela aparecem três operações desse tipo. A linha verde marcada com `POST` representa uma operação de envio de dados para a API.

À direita de cada rota há uma seta. Ao clicar nela, o Swagger expande os detalhes daquela operação. Nessa área expandida, normalmente aparecem os parâmetros aceitos, o corpo da requisição quando houver envio de dados, os possíveis códigos de resposta e um botão chamado "Try it out" (figura 33). Esse botão permite testar a rota diretamente pelo navegador, sem precisar usar ferramentas externas como Postman ou escrever código manualmente.

Mais abaixo, na figura 32, aparece a seção Schemas, a qual mostra os formatos dos objetos usados pela API. Em outras palavras, os schemas descrevem a "forma" dos dados que entram e saem das rotas. Para não sobrecarregar `Program.cs`, os metadados principais do documento ficam em `OpenApi/PetMatchOpenApi.cs`, no projeto `PetMatch.Web`. Essa classe concentra o nome do documento e as informações exibidas no contrato.

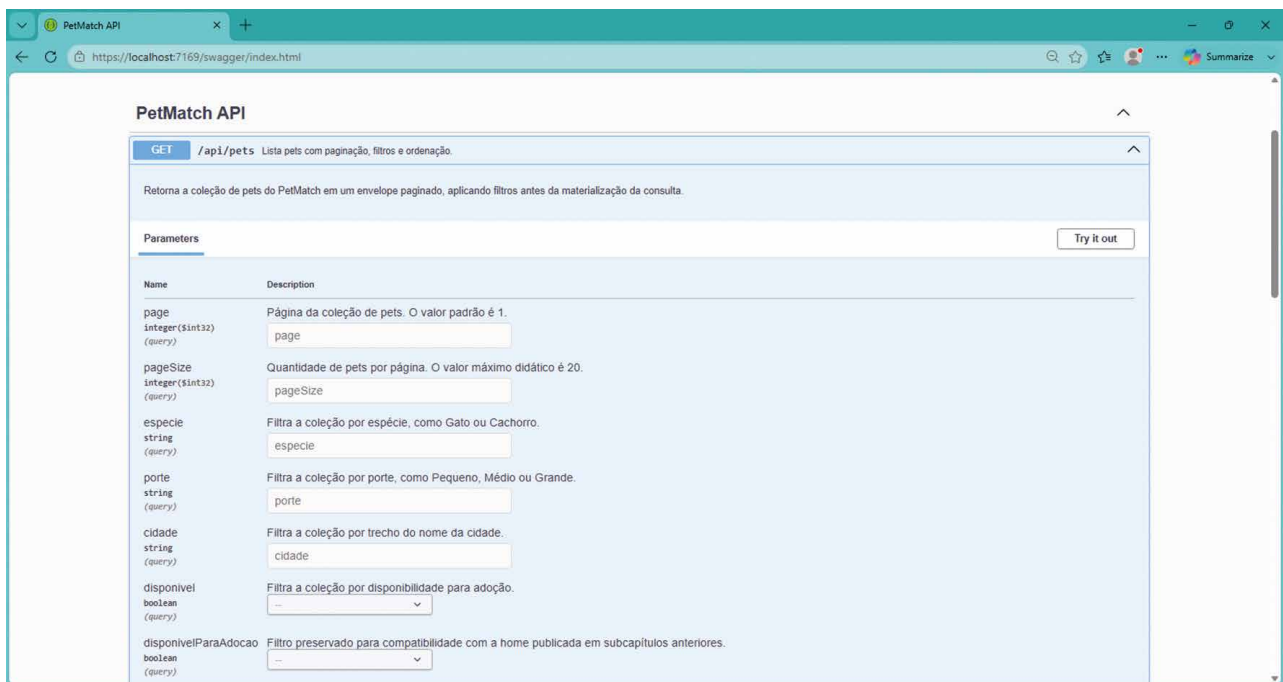


Figura 33 – Interface do Swagger, ao clicar em GET /api/pets. Note o botão "Try it out" à direita

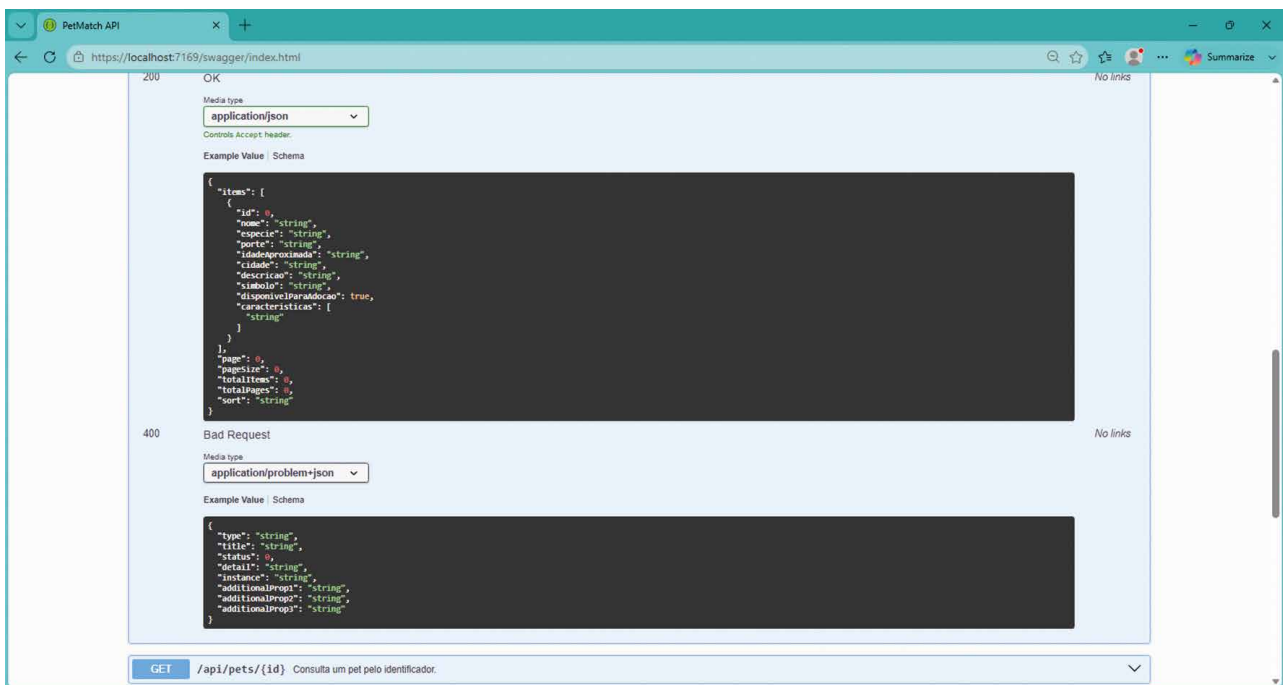


Figura 34 – Rolagem da tela da figura 33, exibindo mais detalhes

```
using Microsoft.OpenApi.Models;

namespace PetMatch.Web.OpenApi;

public static class PetMatchOpenApi
{
    public const string DocumentName = "v1";

    public static OpenApiInfo CreateInfo()
    {
        return new OpenApiInfo
        {
            Title = "PetMatch API",
            Version = "v1",
            Description = "Contrato HTTP do PetMatch para consulta de pets, manifestações de interesse e respostas padronizadas."
        };
    }
}
```

Esse código torna explícito que a API tem nome, versão e finalidade. A versão é relevante porque um contrato não é apenas um retrato informal do presente, mas uma referência para consumidores. Ao versionar o documento, a aplicação prepara um ponto estável de comunicação entre back-end e clientes.

A documentação dos endpoints melhora quando cada rota recebe metadados próprios. No projeto PetMatch.Web, o arquivo `Endpoints/PetEndpoints.cs` foi ajustado com nomes, resumos, descrições e tipos de resposta. O endpoint de listagem de pets, por exemplo, pode ser descrito de forma compatível com paginação, filtros e Problem Details.

```
api.MapGet("/pets", ListPetsAsync)
    .WithName("ListPets")
    .WithSummary("Lista pets disponíveis para adoção.")
    .WithDescription("Retorna uma lista paginada de pets, com filtros por espécie, porte, cidade, disponibilidade e ordenação.")
    .Produces<PagedResponse<PetResponse>>(StatusCodes.Status200OK)
    .ProducesProblem(StatusCodes.Status400BadRequest);
```

Esse trecho mostra que a documentação não depende apenas da assinatura do método. O contrato ganha intenção semântica: a rota recebe um nome estável, o resumo explica a operação, a descrição informa os recursos de consulta e os tipos de resposta indicam o que o cliente pode esperar.

O mesmo cuidado vale para recursos individuais e falhas. O endpoint `/api/pets/{id:int}` preserva o acesso a um pet específico e documenta tanto o sucesso quanto a ausência do recurso.

```
api.MapGet("/pets/{id:int}", GetPetByIdAsync)
    .WithName("GetPetById")
    .WithSummary("Obtém um pet pelo identificador.")
    .WithDescription("Retorna os dados públicos de um pet sem expor informações internas de concorrência.")
    .Produces<PetResponse>(StatusCodes.Status200OK)
    .ProducesProblem(StatusCodes.Status404NotFound);
```

Esse metadado comunica que a API não retorna a entidade persistente completa. O cliente recebe `PetResponse` e detalhes internos, como `RowVersion`, permanecem fora do contrato. Essa escolha foi confirmada também no cliente TypeScript gerado, que não expõe `rowVersion`.

Como o endpoint de manifestações de interesse também devolve respostas estruturadas, o projeto criou contratos específicos em `Contracts: WelcomeResponse, AdoptionInterestResponse` e `AdoptionInterestPetResponse`. A finalidade desses tipos é substituir respostas anônimas por modelos documentáveis. Com isso, Swagger e NSwag conseguem representar melhor o formato das respostas.

```
namespace PetMatch.Web.Contracts;

public sealed record WelcomeResponse(string Mensagem);

public sealed record AdoptionInterestPetResponse(
    int Id,
    string Nome);

public sealed record AdoptionInterestResponse(
    string Mensagem,
    AdoptionInterestPetResponse Pet,
    string Interessado);
```

Esses contratos tornam a API mais previsível. Uma mensagem de boas-vindas, uma manifestação de interesse e a referência ao pet envolvido passam a ter nomes e formas estáveis. Para o front-end, isso reduz suposições sobre o corpo da resposta. Para o documento OpenAPI, isso melhora a clareza dos schemas gerados.

A geração do cliente TypeScript parte do documento OpenAPI produzido pelo projeto. No arquivo `nswag.json`, criado na raiz de `PetMatch.Web`, a configuração usa o documento gerado no build e define a saída `Clients/TypeScript/petmatch-api-client.ts`. O trecho essencial é a leitura do documento e a geração de um cliente com nome próprio.

```
{
  "documentGenerator": {
    "fromDocument": {
      "url": "obj/openapi/v1.json"
    }
  },
  "codeGenerators": {
    "openApiToTypeScriptClient": {
      "className": "PetMatchApiClient",
      "template": "Fetch",
      "output": "Clients/TypeScript/petmatch-api-client.ts"
    }
  }
}
```

A propriedade `"url": "obj/openapi/v1.json"` não aponta para a página visual do Swagger, mas para um arquivo local, criado durante a compilação. Por isso, a geração do cliente TypeScript depende de uma sequência: primeiro, o build produz o documento OpenAPI em `obj/openapi/v1.json`; depois, o NSwag lê esse arquivo e gera o cliente `PetMatchApiClient`.

Dessa forma, o uso de NSwag fica materializado em dois lugares do projeto. O primeiro é o arquivo `nswag.json`, que descreve como o cliente deve ser gerado. O segundo é o arquivo resultante, `Clients/TypeScript/petmatch-api-client.ts`, que aparece após a compilação. Se esse arquivo TypeScript não for criado, a documentação Swagger pode até estar funcionando no navegador, mas a etapa de geração com NSwag ainda não foi concluída corretamente.

Esse arquivo não é uma tela do PetMatch, mas uma configuração de geração. Sua consequência aparece no código produzido: um cliente TypeScript capaz de chamar operações descritas no OpenAPI, com tipos derivados dos contratos reais da API. O arquivo gerado fica em `Clients/TypeScript/petmatch-api-client.ts`, separado da interface MVC e dos arquivos estáticos usados pelo usuário final.

A integração ao build fica em `PetMatch.Web.csproj`. Essa etapa precisa explicitar duas responsabilidades diferentes. A primeira é gerar o documento OpenAPI físico, em `obj/openapi/v1.json`. A segunda é executar o NSwag a partir desse documento para criar o cliente TypeScript. Sem a primeira etapa, o arquivo indicado em `nswag.json` não existe; sem a segunda, o contrato existe, mas nenhum cliente TypeScript é produzido.

Para permitir a geração do documento OpenAPI durante a compilação, o projeto deve incluir o suporte de geração em tempo de build. Em projetos ASP.NET Core recentes, isso é feito com o pacote `Microsoft.Extensions.ApiDescription.Server`, que permite produzir documentos OpenAPI no processo de build. A propriedade `OpenApiGenerateDocuments` habilita essa geração, e `OpenApiDocumentsDirectory` define onde o arquivo será gravado.

```
<PropertyGroup>
  <OpenApiGenerateDocuments>true</OpenApiGenerateDocuments>
  <OpenApiDocumentsDirectory>obj/openapi</OpenApiDocumentsDirectory>
  <OpenApiGenerateDocumentsOptions>--document-name v1 --file-name v1
</OpenApiGenerateDocumentsOptions>
</PropertyGroup>
```

Com essa configuração, o projeto passa a gerar o arquivo `obj/openapi/v1.json` durante a compilação. Esse arquivo é a entrada usada pelo `nswag.json`.

Depois da geração do documento, o alvo `GenerateTypeScriptClient` executa o NSwag. O pacote `NSwag.MSBuild` disponibiliza comandos que podem ser chamados pelo MSBuild, como `$(NSwagExe_Net90)`, permitindo que a geração do cliente seja incorporada ao ciclo normal de build do projeto.

```
<Target Name="GenerateTypeScriptClient" AfterTargets="GenerateOpenApiDocuments">
  <Error
    Condition="!Exists('obj/openapi/v1.json')"
    Text="O documento OpenAPI obj/openapi/v1.json não foi encontrado. Verifique a
    geração OpenAPI em tempo de build antes de executar o NSwag." />

  <Exec Command="$ (NSwagExe_Net90) run nswag.json" />
</Target>
```

O `GenerateOpenApiDocuments` produz o contrato OpenAPI. Em seguida, o NSwag lê o arquivo configurado em `nswag.json` e gera `Clients/TypeScript/petmatch-api-client.ts`. Assim, o NSwag não aparece na tela `/swagger`, mas no processo de build e no arquivo TypeScript gerado. Esse trecho transforma a geração de cliente em parte do ciclo normal de compilação. A decisão é deliberada: se o contrato muda, o cliente TypeScript deve acompanhar essa mudança. O uso de NSwag pelo build preserva a fonte de verdade no OpenAPI gerado pelo próprio projeto.

O resultado esperado é uma separação clara. No navegador, a home continua voltada ao usuário final, com links para ver e cadastrar pets, sem botão para API. A página `/Pets` continua exibindo cards reais. Em desenvolvimento, `/swagger` permite consultar o contrato e `/swagger/v1/swagger.json` fornece o documento usado para geração. No gerenciador de soluções, o arquivo `Clients/TypeScript/petmatch-api-client.ts` aparece após a compilação. Os pacotes adicionados neste tópico foram `Swashbuckle.AspNetCore`, para expor a documentação interativa em desenvolvimento; `Microsoft.Extensions.ApiDescription.Server`, para gerar o documento OpenAPI durante o build; e `NSwag.MSBuild`, para gerar o cliente TypeScript a partir desse contrato.



Resumo

Nesta unidade, o texto parte da distinção entre memória da aplicação e banco relacional. Nesse contexto, o tópico apresenta o mapeamento objeto-relacional como mediação entre objetos C# e estruturas relacionais, conferindo ao Entity Framework Core papel central por permitir que entidades, relacionamentos e regras de persistência sejam descritos no próprio código. Ao mesmo tempo, situa ADO.NET e Dapper como alternativas relevantes em cenários que exigem maior controle sobre SQL, conexões, comandos, leitores de dados e desempenho de consultas específicas, mostrando que a escolha da tecnologia de acesso a dados envolve equilíbrio entre produtividade, controle, legibilidade e complexidade.

Na parte prática, o tópico 3 demonstra como o PetMatch passa a usar SQLite com EF Core, por meio da criação da entidade persistente `Pet`, do `PetMatchDbContext`, de migrations, seed e configuração de pooling de conexões e de contexto. A aplicação preserva suas rotas MVC e Minimal APIs, mas altera a fonte dos dados: os pets deixam de existir apenas em uma coleção temporária e passam a ser consultados e gravados em uma base persistente compartilhada.

O tópico 4 aborda a construção de APIs REST como uma camada de integração baseada em contratos HTTP estáveis, deslocando o foco de simples respostas em JSON para uma superfície de comunicação previsível, documentável e reutilizável. Nesse sentido, aparecem como elementos centrais os DTOs, os parâmetros de consulta, a paginação, os filtros e a ordenação, que tornam a listagem de recursos mais controlada, eficiente e compreensível para clientes externos. O tópico também destaca que os erros fazem parte do contrato da API, razão pela qual respostas improvisadas devem ceder lugar a formatos padronizados, como Problem Details, capazes de expressar falhas com status HTTP, título e detalhes de maneira consistente.

Na segunda parte, o tópico avança da implementação da API para documentação e reutilização por ferramentas. A OpenAPI é apresentada como especificação formal do contrato, enquanto o Swagger é tratado como interface e conjunto de ferramentas para visualizar e testar esse contrato no navegador, preferencialmente em ambiente de desenvolvimento. O texto diferencia essa documentação interativa da geração de clientes: o NSwag utiliza o documento OpenAPI como fonte para produzir código TypeScript, reduzindo duplicações manuais entre front-end e back-end e preservando maior alinhamento entre rotas, parâmetros, tipos de resposta e formatos de erro.



Exercícios

Questão 1. Uma instituição desenvolveu, com ASP.NET Core, Entity Framework Core e SQLite, uma plataforma para cadastrar animais disponíveis para adoção. Na primeira versão, os registros eram mantidos em uma coleção em memória e desapareciam quando a aplicação era encerrada.

Para tornar os dados duráveis, a equipe adotou a seguinte configuração:

- a connection string foi definida como `Data Source=adocoes.db;Pooling=True;`
- o contexto foi registrado com `AddDbContextPool<AdocaoDbContext>(...)` e configurado para utilizar a connection string do SQLite;
- a entidade `Animal` foi exposta por meio de um `DbSet<Animal>`;
- durante a inicialização, a aplicação executa `Database.Migrate()`;
- em seguida, um serviço de seed verifica `Animais.Any()` e insere registros iniciais somente quando a tabela está vazia;
- o controller MVC e os endpoints Minimal API recebem instâncias de `AdocaoDbContext` configuradas para acessar o mesmo arquivo SQLite;
- as consultas destinadas apenas à exibição utilizam `AsNoTracking()` e projetam os resultados para objetos de resposta;
- os cadastros são gravados com `SaveChangesAsync()`.

Após a alteração, um animal cadastrado pela área MVC continuou aparecendo depois que a aplicação foi encerrada e iniciada novamente. O mesmo registro também passou a ser retornado pela API.

Considerando os mecanismos empregados, assinale a alternativa que explica corretamente esse comportamento.

- A) A durabilidade decorre do arquivo SQLite e de `Database.Migrate()`, que grava automaticamente as entidades rastreadas antes do encerramento. `SaveChangesAsync()` apenas atualiza o estado dos objetos dentro do `DbContext` e não participa diretamente da escrita no banco.
- B) `SaveChangesAsync()` envia as alterações ao arquivo SQLite. `Database.Migrate()` atualiza o esquema, o seed insere dados somente quando a tabela está vazia e `AsNoTracking()` modifica apenas o rastreamento das consultas. O pooling reaproveita recursos, enquanto MVC e API acessam a mesma fonte persistente por meio de contextos distintos.

- C) `AddDbContextPool` conserva as alterações pendentes quando uma instância é devolvida ao pool e as grava quando essa instância é reutilizada. Por esse motivo, o cadastro pode permanecer no banco mesmo que o código não execute `SaveChangesAsync()` antes do fim da requisição.
- D) `AsNoTracking()` obriga cada consulta a recarregar os registros diretamente do arquivo e constitui o principal mecanismo pelo qual MVC e API observam os mesmos dados. Sem essa opção, cada interface manteria uma cópia independente das entidades consultadas.
- E) Como o controller e os endpoints recebem `AdocaoDbContext`, o contêiner fornece uma única instância compartilhada durante toda a execução. Após uma reinicialização, essa mesma instância é reconstruída com os objetos que estavam rastreados antes do encerramento.

Resposta correta: alternativa B.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: `Database.Migrate()` trata da estrutura do banco, aplicando migrations pendentes. Ele não confirma inclusões ou alterações realizadas pelas operações da aplicação. Essas modificações são convertidas em comandos para o banco quando `SaveChanges()` ou `SaveChangesAsync()` é executado.

B) Alternativa correta.

Justificativa: a permanência do registro depois do encerramento demonstra que os dados foram transferidos da memória do processo para o arquivo do banco. Cada requisição pode utilizar um contexto diferente e ainda consultar as mesmas linhas, pois todos os contextos estão configurados para a mesma fonte. O rastreamento e o pooling modificam o modo de uso dos objetos e dos recursos, mas não substituem a confirmação da transação.

C) Alternativa incorreta.

Justificativa: uma instância devolvida ao pool deve ser reinicializada para outro uso e não funciona como uma área de alterações pendentes. O retorno ao pool não dispara automaticamente a gravação das entidades. Se `SaveChangesAsync()` não for executado, as inclusões existentes apenas no contexto não se tornarão duráveis.

D) Alternativa incorreta.

Justificativa: `AsNoTracking()` informa que as entidades lidas não precisam permanecer no rastreador de alterações. Com ou sem essa opção, a consulta pode acessar o mesmo banco. A opção é adequada para leituras que não serão modificadas, mas não cria compartilhamento entre interfaces nem participa da escrita.

E) Alternativa incorreta.

Justificativa: o DbContext normalmente tem o ciclo de vida associado a um escopo, como uma requisição. O pooling permite reutilizar instâncias já reinicializadas, mas não cria um singleton com entidades permanentes. Depois que o processo termina, somente o que foi gravado no banco pode ser recuperado por novas consultas.

Questão 2. Uma universidade desenvolveu uma API HTTP para disponibilizar seu catálogo de publicações científicas a portais institucionais e aplicações móveis. O endpoint GET /api/publicacoes aceita filtros por área e ano, ordenação por título ou data, além dos parâmetros page e pageSize. A equipe definiu o limite máximo de 50 itens por página.

A resposta da listagem utiliza o seguinte contrato:

```
public sealed record PagedResponse<T>(
    IReadOnlyList<T> Items,
    int Page,
    int PageSize,
    int TotalItems,
    int TotalPages);
```

Os dados públicos de cada publicação são representados por PublicacaoResponse, que não contém propriedades internas da entidade persistente. Quando uma publicação não é encontrada, o endpoint retorna o status HTTP 404 e um corpo no formato Problem Details, com campos como title, status e detail.

A aplicação também tem as seguintes configurações:

- o Swagger apresenta a documentação interativa em /swagger, apenas no ambiente de desenvolvimento;
- o contrato OpenAPI pode ser consultado em /swagger/v1/swagger.json durante a execução;
- durante a compilação, um documento OpenAPI é gravado em obj/openapi/v1.json;
- o NSwag lê esse arquivo e gera Clients/TypeScript/publicacoes-api-client.ts;
- os filtros, a ordenação, Skip e Take são aplicados antes de ToListAsync().

Durante uma revisão técnica, a equipe discutiu como preservar a clareza do contrato, a eficiência das consultas e a separação entre a interface destinada aos usuários e as ferramentas destinadas ao desenvolvimento.

Assinale a alternativa que apresenta a decisão tecnicamente adequada.

- A) Utilizar `PublicacaoResponse`, `Problem Details` e `Swagger` somente no desenvolvimento preserva o contrato externo. Como `pageSize` está limitado a 50, a aplicação pode executar `ToListAsync()` antes dos filtros e aplicar a paginação em memória sem alterar o volume transferido do banco.
- B) Os filtros, a ordenação e a paginação devem ser aplicados antes de `ToListAsync()`. Os endpoints devem utilizar contratos próprios. O cliente `TypeScript`, porém, deve ser gerado a partir de `/swagger/index.html`, pois essa página reúne a descrição visual dos schemas e das operações.
- C) O documento `OpenAPI` em `JSON` pode alimentar o `NSwag`. Os contratos próprios evitam a exposição da entidade. Para uniformizar as respostas, todos os erros devem usar o status `HTTP 200`, mantendo o código real apenas na propriedade `status` do corpo `Problem Details`.
- D) Os endpoints devem empregar contratos próprios, aplicar filtros, ordenação, `Skip` e `Take` antes da materialização e devolver erros com códigos `HTTP` adequados e `Problem Details`. O `Swagger UI` pode permanecer restrito ao desenvolvimento, enquanto o `NSwag` utiliza o documento `OpenAPI` produzido durante o build.
- E) Os filtros e a paginação devem ocorrer no banco. O cliente deve ser gerado pelo documento `OpenAPI`. O endpoint de detalhe, contudo, pode devolver diretamente a entidade persistente, pois a documentação gerada impede que novas propriedades internas sejam incorporadas ao contrato.

Resposta correta: alternativa D.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: o limite de 50 define quantos itens serão devolvidos depois da paginação. Se `ToListAsync()` for chamado antes dos filtros, todos os registros correspondentes à consulta inicial poderão ser transferidos para a memória. A limitação da resposta não reduz o volume já lido do banco.

B) Alternativa incorreta.

Justificativa: o `Swagger UI` interpreta o documento `OpenAPI` e cria uma interface para leitura e testes. O gerador de clientes precisa consumir a especificação estruturada em `JSON` ou `YAML`, na qual estão descritos os caminhos, os parâmetros, as respostas e os schemas.

C) Alternativa incorreta.

Justificativa: o corpo `Problem Details` complementa o código `HTTP`; ele não o substitui. Bibliotecas, proxies e clientes utilizam o status da resposta para distinguir sucessos, erros do solicitante e falhas do servidor. Retornar 200 para um recurso inexistente produz uma indicação contraditória no protocolo.

D) Alternativa correta.

Justificativa: enquanto a consulta permanece como IQueryable, o provedor pode traduzir os filtros, a ordenação e a paginação para SQL e recuperar somente o conjunto necessário. Os objetos de resposta definem explicitamente o formato público. O arquivo OpenAPI, por ser uma representação estruturada desse formato, pode ser usado de maneira reproduzível na geração do cliente.

E) Alternativa incorreta.

Justificativa: o OpenAPI descreve o tipo efetivamente exposto pelo endpoint. Se a entidade for utilizada como resposta, suas propriedades públicas podem aparecer no schema e alcançar os consumidores. O documento não bloqueia alterações futuras no modelo persistente nem cria uma separação automática entre o banco e o contrato externo.

This image shows a single sheet of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.